

Design and Implementation of an FPGA- and RK3588-Based Image Classification and Processing System

Xiaoping Yuan

Qiyang Tu

School of Information and Control Engineering, China University of Mining and Technology,
Xuzhou, Jiangsu 221116, China

Abstract: To address the low execution efficiency and long latency of the Vision Transformer (ViT) model on CPU-based platforms, this study designs an image acquisition and processing system based on a field-programmable gate array (FPGA) and a neural network processing unit (NPU), hereafter referred to as the “FPGA system.” First, the system exploits the stability of FPGA timing control and computation to match the pixel-transfer bandwidth of the image sensor interface. The patch-embedding layer is decomposed into vector-based computations, and a pipelined design accumulates and outputs tensors after fusion of the model’s additive terms. Once the patch-embedding results are obtained, position flags are appended to the outputs to form position-identification packets. Next, a User Datagram Protocol (UDP) stack transmits the computational results of the patch-embedding neural network structure from the FPGA, while image acquisition, video display, and UDP transmission operate concurrently. Finally, an RK3588 NPU-based implementation incorporates multithreading and a thread pool, UDP reception, and thread-pool processing of attention computation tasks. After partitioning the design workflow, verifying the design, and performing optimization and debugging, the average execution time per frame is reduced from an initial 0.6439 s to 0.0995 s. The FPGA system uses 56.42% of its block RAM resources and exhibits no timing violations. The mean recognition accuracy is 0.85 for pedestrians and 0.905 for vehicles.

Keywords: FPGA; RK3588; vector-based computation; additive-term fusion; position-identification packets; multithreaded design

Introduction

With the rapid development of computer vision technology, Transformer-based vision models, such as the Vision Transformer (ViT), are gradually replacing convolutional neural networks (CNNs) [1, 2]. ViT models capture global contextual information through self-attention and have demonstrated significant advantages in tasks such as image classification and object detection [3, 4]. However, their high computational complexity and large parameter counts pose challenges in embedded applications with stringent real-time requirements, such as autonomous driving and robot vision [5, 6]. With their high parallelism, low power consumption, and reconfigurability, FPGAs have become important hardware platforms for accelerating deep learning models [7, 8]. For example, FPGA-based real-time image preprocessing algorithms have enabled efficient edge detection and distortion correction in robot vision systems [9, 10].

FPGA acceleration designs are also widely used for CNNs [11, 9, 12]. However, these models have large parameter counts, while on-chip digital signal processor (DSP) and random access memory (RAM) resources are constrained [13, 14]. Off-chip storage resources or heterogeneous systems are therefore required to handle computational tasks [15]. Communication between processors in a heterogeneous system introduces additional overhead, affecting the real-time performance of these tasks [16]. Consequently, investigating ViT optimization and real-time applications on FPGAs and embedded sys-

tems is key to overcoming model deployment challenges in computationally constrained settings [17, 18].

1 System Architecture and Design Workflow

1.1 System Architecture

Figure 1 shows the architecture of the image acquisition and processing system based on an FPGA and a neural network processing unit (NPU), hereafter referred to as the “FPGA system.” To improve design efficiency, the embedding layer, `class_token` concatenation, and positional embedding operations are deployed on the FPGA, while the embedded NPU in the RK3588 performs the computations of the algorithm’s deeper network layers. A User Datagram Protocol (UDP) stack [19] transfers the computational data to the embedded NPU side for processing. Multithreading and queue buffering enable real-time data reception and address the cumulative storage and computation latency caused by faster writes than reads in the queue. The remaining network layers are computed on the embedded side.

To facilitate observation of image acquisition and recognition, the acquired images are output through a High-Definition Multimedia Interface (HDMI). The embedded side transmits computational data through a universal asynchronous receiver/transmitter (UART) to a microcontroller unit (MCU), which executes other tasks

and displays the classification results. Table 1 lists the functional and performance requirements of the subsystems.

1.2 Design Workflow

Figure 2 illustrates the design workflow of the FPGA system. The system must synchronously drive the camera, acquire images, and display video; therefore, the UDP packets and algorithms must be designed as part of the system design. After algorithm training and inference, the software algorithm team and the FPGA team must jointly clarify the forward-propagation computation flow. The algorithm is tuned through fixed-point conversion to suit the computational characteristics of the FPGA system, and algorithmic structures that are unfavorable for FPGA implementation are optimized.

The model is then partitioned into shallow and deep networks according to the workloads of the FPGA and the RK3588/NPU. The shallow network to be deployed on the FPGA undergoes operator-level analysis of its operations and resource evaluation. After the acquisition, transmission, and display tasks have been implemented, the FPGA model-processing architecture is modeled according to the timing and bandwidth of the transmission and acquisition interfaces. The device is selected based on the resource requirements of the algorithm-level model and the interface system. The clock, data, control, and reset architectures for implementing the algorithm are then designed according to the total quantity and distribution of device resources and the device's electrical characteristics. Each unit is functionally verified, and the top-level design is integrated after verification passes.

To determine whether the system architecture produces correct processing results, the timing and reset behavior of the camera image interface must be simulated, and text-based simulation stimuli must be prepared. In the PyTorch framework, the image is resized to 1024 px $\times$ 768 px, and the processed image data are saved to a text file. The FPGA development software reads this `.txt` file, parses its contents into binary data, and stores them. Pixel information is read from the text according to the interface timing logic of the OV5640 image sensor and passed to the processing unit. The processing unit's output is saved to another text file, which is read in PyTorch and converted into decimal data. These decimal data are used as the input to the deep network. Image classification accuracy obtained using the FPGA processing unit is compared with that obtained using PyTorch. A smaller absolute difference between the two accuracy values indicates greater correctness of the overall processing architecture and a smaller computational error relative to the theoretical implementation.

After functional verification is completed without errors, the FPGA system undergoes synthesis, which converts the Verilog-described design into a gate-level

netlist, and placement and routing, which maps logic elements such as logic gates and storage units to physical locations on the FPGA according to the design's logic netlist. Code-level resource optimization and timing optimization are then performed to address timing violations in the placement-and-routing results, namely failures to meet flip-flop setup and hold times. On-board verification follows once optimization is complete.

A Gigabit Ethernet reception program is written on the PC, and the FPGA's computational results are transferred to the PC over Gigabit Ethernet to verify the computation performed by the Transformer Encoder neural network structure in the ViT algorithm. If problems occur, the correctness of the verification program and the PC program is checked. Once verification succeeds, model conversion and task-scheduling design are carried out for the RK3588, and the deep network is deployed on the NPU for performance evaluation. Following NPU optimization, system design and development are complete.

2 ViT Model Structure, Program Overview, and Parameter Fixed-Point Quantization

2.1 Model Network Structure

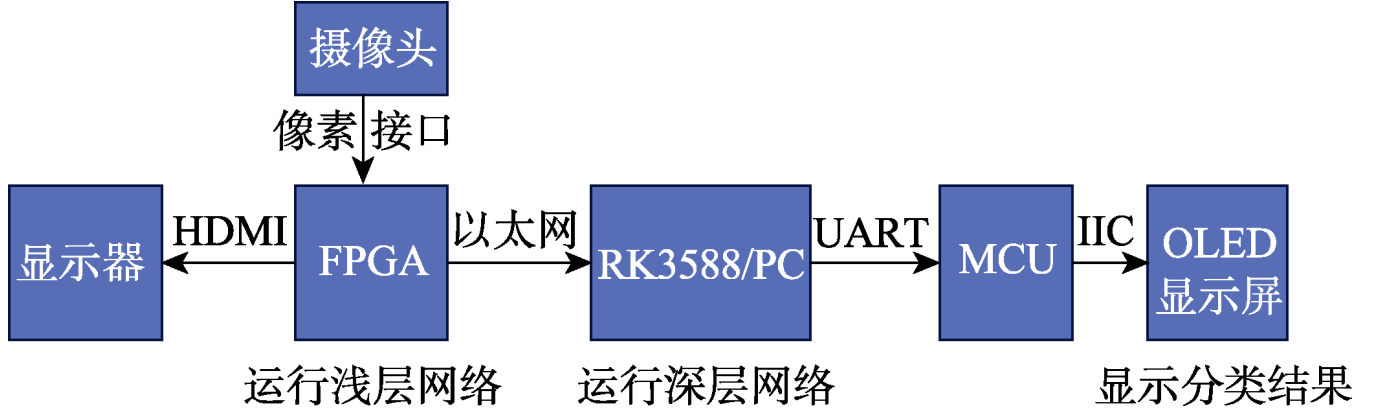
Figure 3 shows the ViT model's network structure [1]. The complete forward-propagation network comprises an Embedding layer, a Transformer Encoder layer, and an MLP Head layer. The model input has dimensions [1, 3, 224, 224]. This architecture relies on the attention mechanism and uses global features to classify images, thereby improving image classification accuracy.

2.2 Forward-Propagation Program Overview

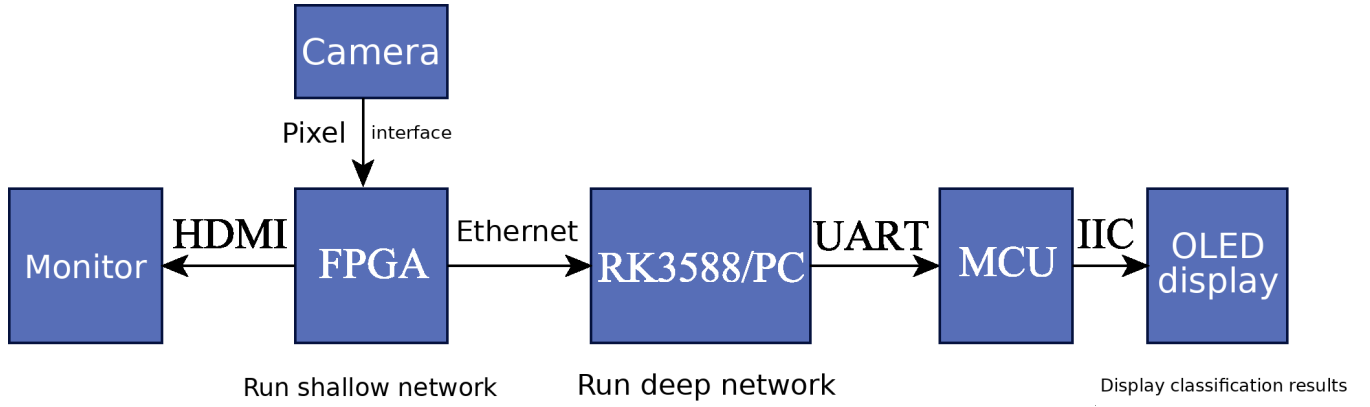
Figure 4 presents the framework of the ViT model's forward-propagation program. Convolution transforms the input tensor into a tensor of dimensions [1, 14, 14, 768]. After dimensionality reduction, `class_token` concatenation, and positional embedding, the data dimensions become [1, 197, 768]. The data are then fed into the Transformer Encoder layer, followed by normalization (`Norma_layer`) and a fully connected structure (`Linear`), to produce the output classification.

2.3 Parameter Fixed-Point Quantization and Embedding Layer Evaluation

For FPGA-suitable quantization, the model parameters used in computation are binary fixed-point numbers. All model parameter structures must therefore be quantized to an appropriate binary bit width. After quantization,

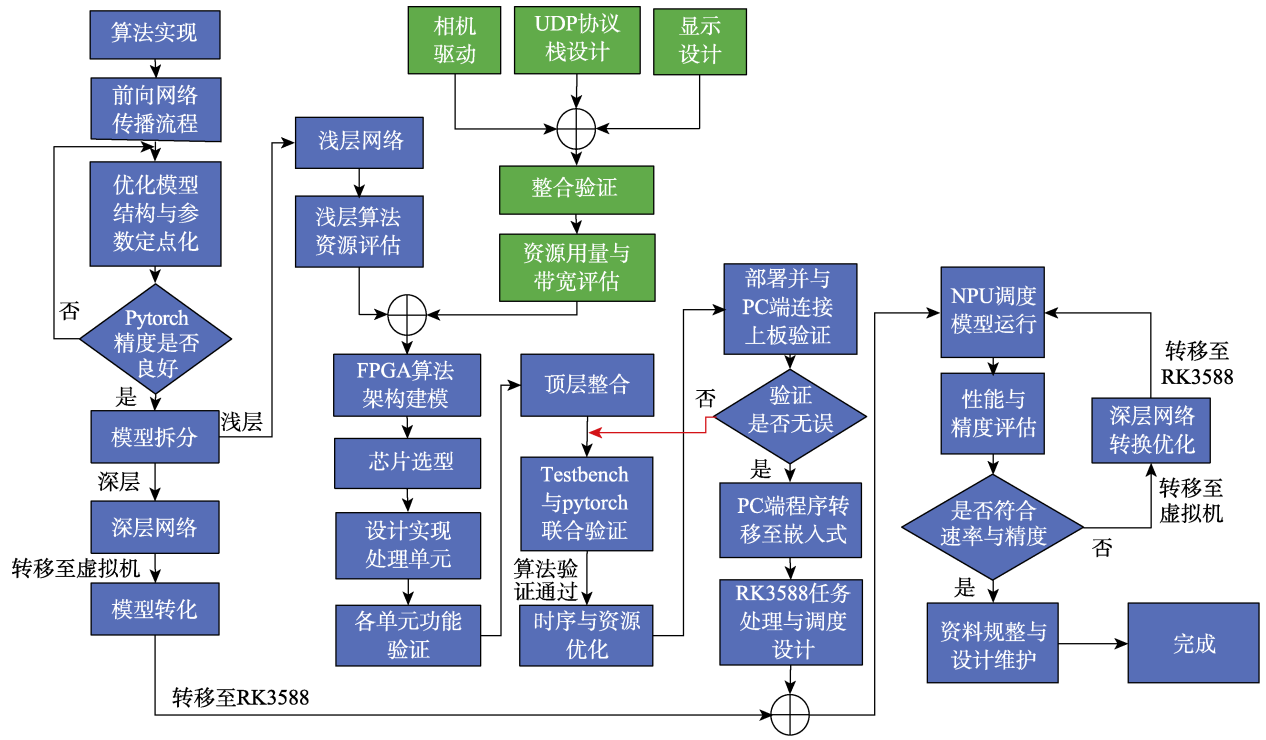


(a) Original source

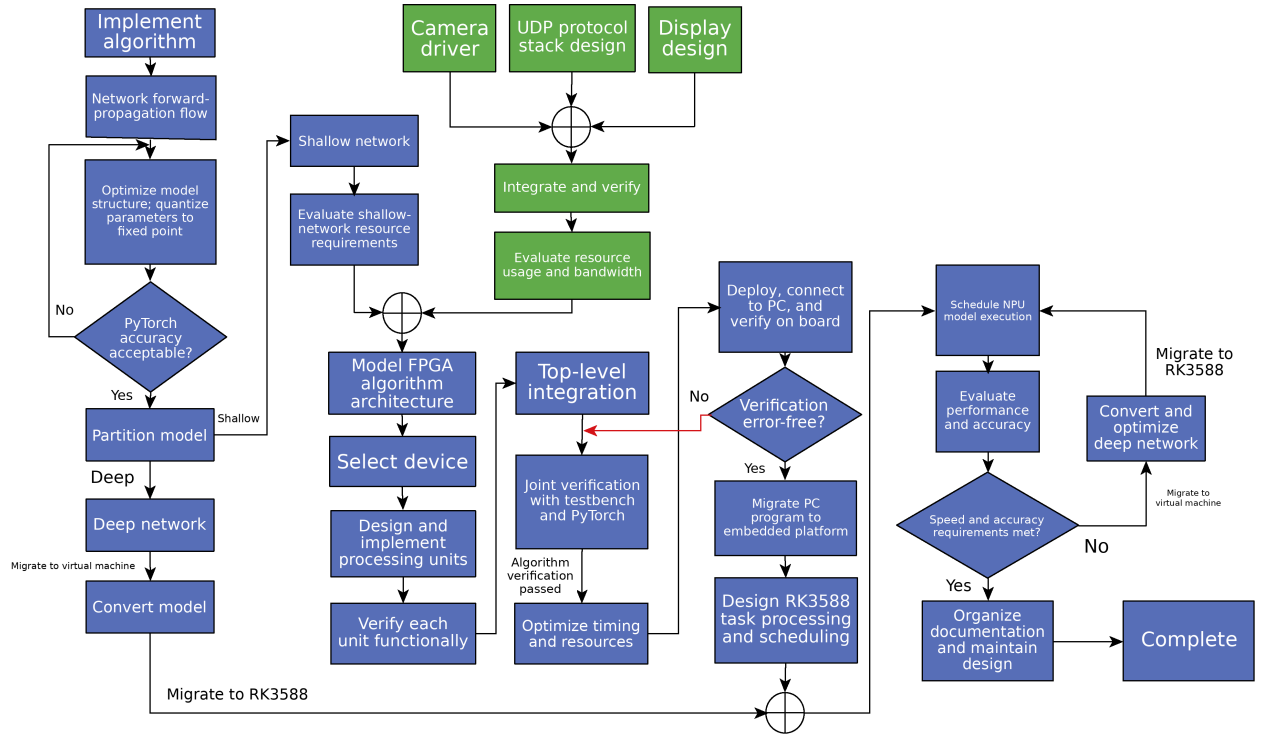


(b) English version

Figure 1: Architecture of the FPGA system. IIC denotes inter-integrated circuit.



(a) Original source



(b) English version

Figure 2: Design workflow of the FPGA system. The symbol $\oplus$ indicates waiting until all tasks have completed.

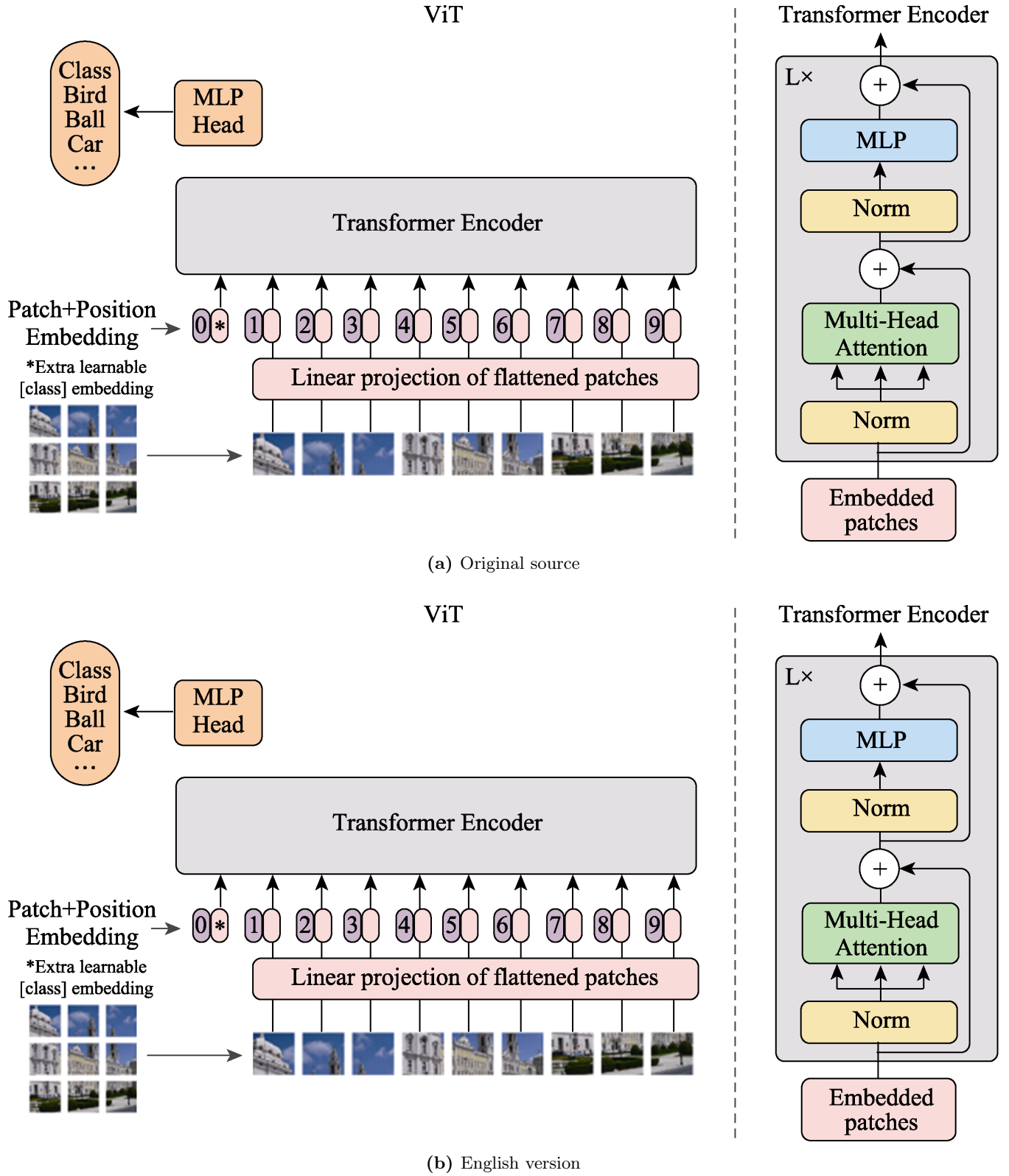


Figure 3: Network structure underlying the algorithm. The source diagram already uses English labels.

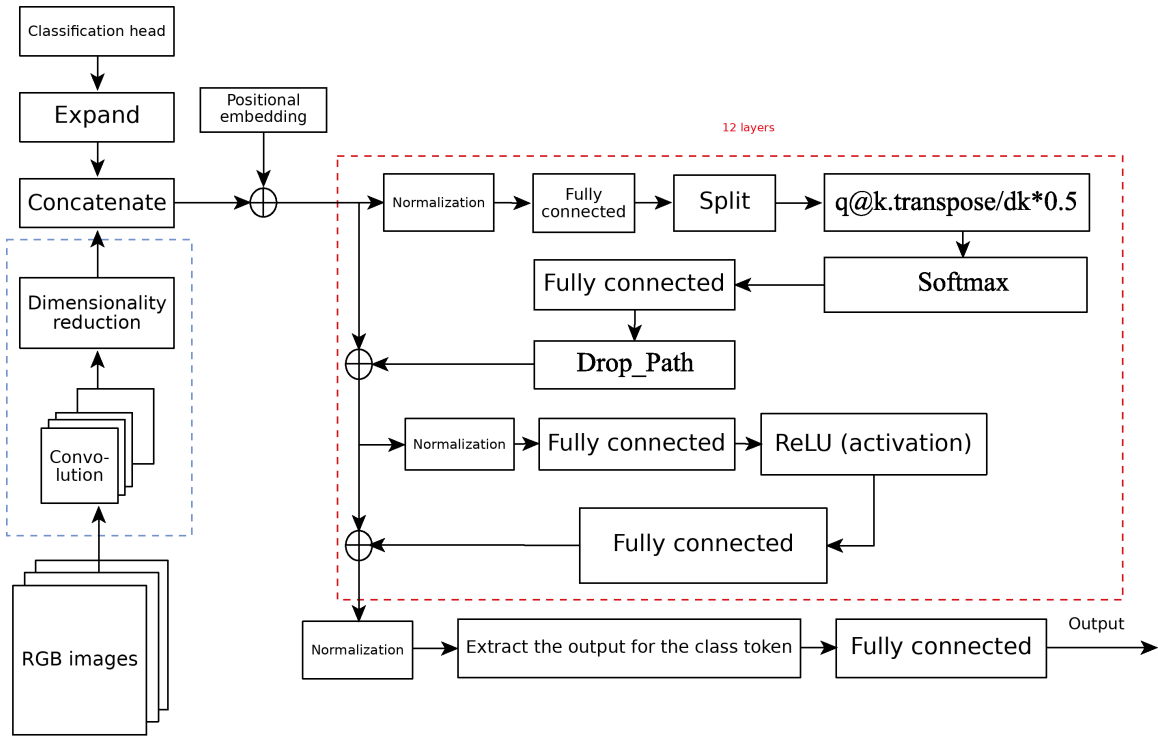
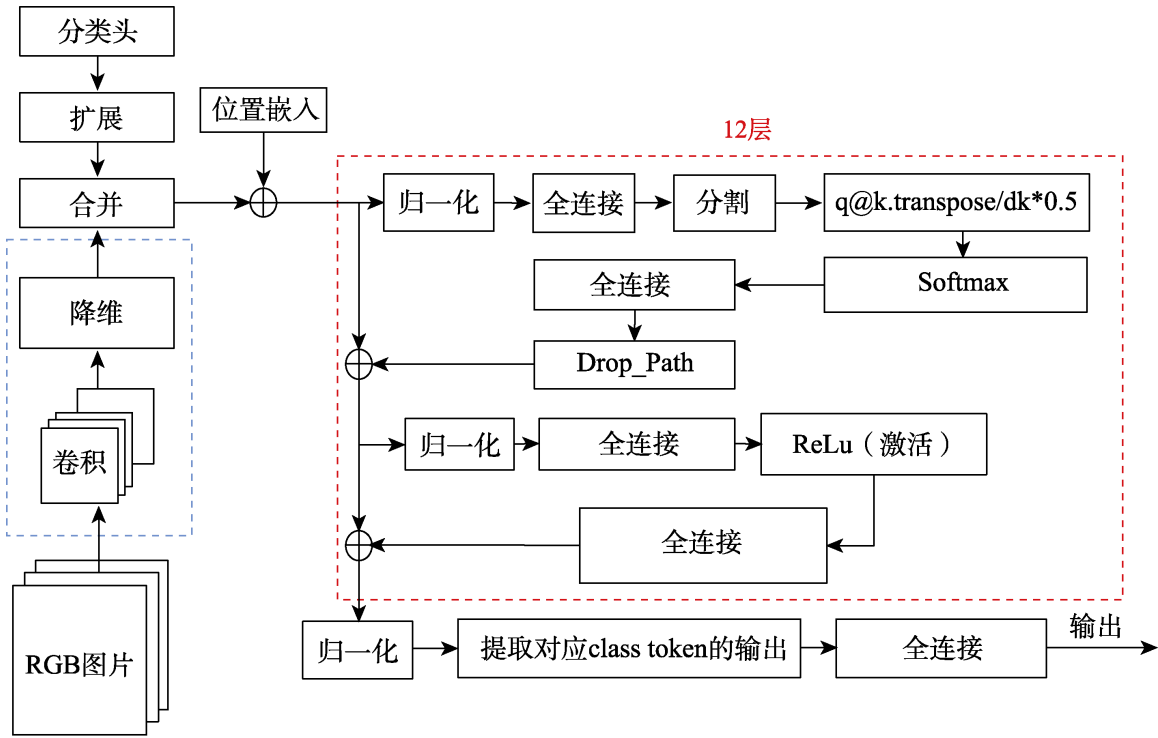


Figure 4: Framework of the forward-propagation program.

Table 1: Functional and performance requirements of the subsystems.

Subsystem	Functional tasks	Performance requirements
FPGA system	Acquire and display images; perform preprocessing and compute the shallow ViT network; transmit computational results.	Complete patch-embedding computation and transmission with the camera operating at a frame rate of 60 Hz.
RK3588/PC	Receive computational results; compute the deep ViT network; output computational results through a serial port.	Receive data without frame loss; achieve a model computation frame rate of 60 Hz.
MCU	Receive and display classification results; execute other control tasks.	None.

Table 2: Embedding layer parameter analysis.

Parameter name	Tensor dimensions
<code>cls_token</code>	[1, 1, 768]
<code>pos_embed</code>	[1, 197, 768]
<code>patch_embed.proj.weight</code>	[768, 3, 16, 16]
<code>patch_embed.proj.bias</code>	[768]

the parameters are converted to binary and stored in the FPGA’s internal block random access memory (BRAM).

A smaller quantization bit width requires fewer FPGA logic resources. Suppose that an original floating-point model parameter is b and that fixed-point arithmetic uses one sign bit, one integer bit, and n fractional bits. First, the function `np.clip(b, -a, +a)` constrains the original parameter b to the interval $[-a, +a]$:

$$c = \text{np.clip}(b, -a, +a), \quad a = 1 - 0.5^n.$$

Here, c is the result of applying `np.clip()` to the original parameter b , and `np.clip()` is the clipping function in the NumPy library.

To determine the specific value of n for fixed-point quantization, the quantization function is applied after model training and before prediction, quantizing the network parameters layer by layer. A smaller value of n is determined by repeatedly modifying n during the training process. Figure 5 shows the parameter fixed-point quantization program flow.

The Embedding layer contains a total of 742,656 network parameters, as analyzed in Table 2. Once the Embedding layer has been evaluated, the computational architecture can be designed based on the model parameters and computational workload, in conjunction with the system’s transmission interfaces.

3 FPGA Architecture Design and Device Selection

3.1 Camera Interface Analysis

The FPGA system uses an OV5640 image sensor [20]. After the sensor driver has been configured, the sensor’s

output interface timing is as shown in Figure 6. Each image frame is controlled by the vertical synchronization (VSync) signal: a high-level pulse indicates the start of a new frame, while the pixel information for that frame is transmitted during the low-level interval. The horizontal reference (HREF) signal controls the transmission of pixel information for an entire row. A high level indicates that the image information transmitted on the clock pulses is valid. The design uses an image frame rate of 60 Hz and a resolution of $1024 \text{ px} \times 768 \text{ px}$.

3.2 Preprocessing Unit Design

The model’s input image is a $224 \text{ px} \times 224 \text{ px}$ tensor with pixel values in the interval $[-1, 1]$. Because the camera captures images at $1024 \text{ px} \times 768 \text{ px}$, the data must be resized and normalized before they can be used as input.

During normalization, two binary zeros can be prepended to the most significant bit of the unsigned 8-bit pixel data. These two bits represent the sign bit and the integer bit, respectively. Because the pixel values are in RGB565 format, zeros are appended at the least significant end to extend each value to 8 bits; consequently, the last two bits are always zero. After adding the signed two’s-complement representation of -0.5 , the values lie in the interval $(-0.5, 0.5)$, and the three most significant bits have identical values. The most significant bit and the final two bits can therefore be discarded. The resulting data are used as a normalized array of signed values with six fractional bits. Figure 7 illustrates the FPGA data normalization operation.

3.3 Vector Convolution Unit Design

After image preprocessing, the resulting pixel-valid timing is as shown in Figure 8. One row is sampled from every three consecutive rows of the original image. Within each valid row, one valid pixel is output after every eight consecutive pixel-clock cycles.

To use the algorithm model’s input data efficiently, the normalized input image data can be processed using a single-row vector of the convolution kernel. The data transmitted serially by the camera are stored sequentially, in input order, starting at the initial address

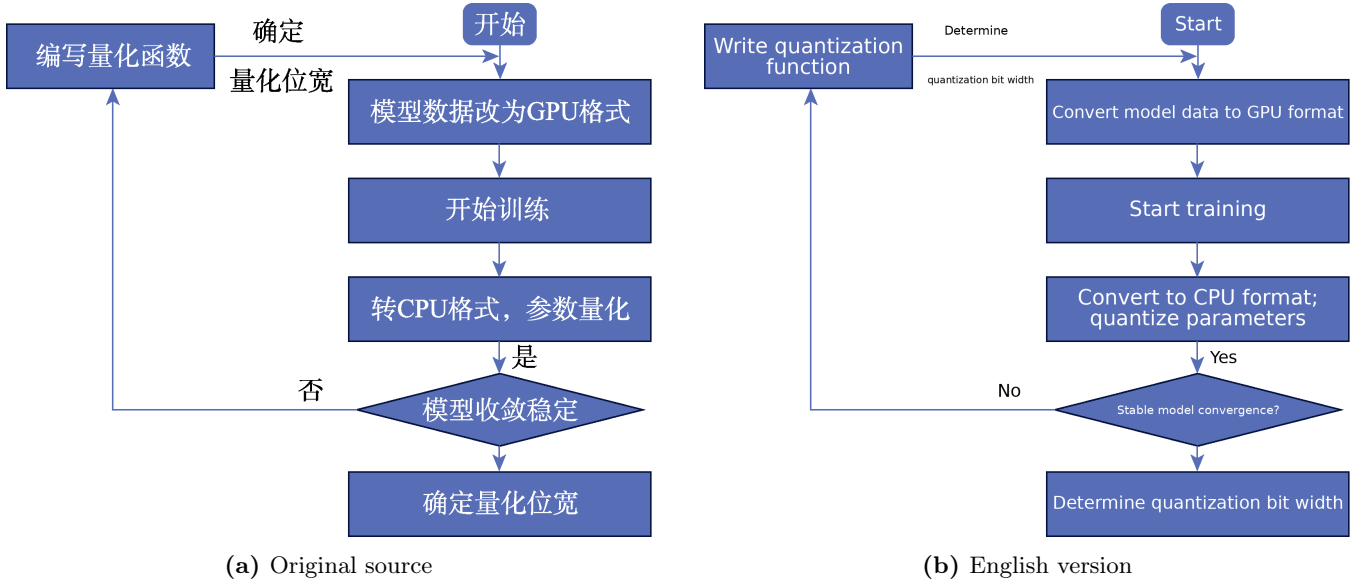


Figure 5: Program flow for parameter fixed-point quantization.

of a dual-port RAM.

To ensure that the input data and weight positions match during convolution, the original model parameters are reordered and then stored in read-only memory (ROM). For BRAM initialization, the model parameters must be written to a .coe file in a signed binary format with one integer bit. The parameter storage order in the weight ROM must follow the input data sequence at successively increasing addresses. Its contents correspond to the weights required by the input data during convolution. The input/output behavior of this processing unit can be abstracted as the convolution unit I/O bit-order diagram in Figure 9. If a convolution-kernel window position required in the next clock cycle has not yet been covered, its corresponding weights are concatenated and stored at a new address. This process continues until all positions in the convolution-kernel window have been covered.

When modeling the overall processing architecture, resource and timing constraints must be calculated. With a convolution-kernel width of 16 and a buffer holding data spanning two convolution-kernel widths, the interval required to complete data buffering must be determined. Assuming a pixel-clock frequency of 48 MHz, the valid-signal timing in Figure 8 gives the buffering interval T_{buffer} as

$$T_{\text{buffer}} = \frac{1}{48} \times 32 \times 4 \times 2. \quad (1)$$

To determine the convolution clock period while maintaining reasonable FPGA resource usage, let h denote the computational depth of the input vector processed per cycle, and let f denote the computing unit's operating frequency in MHz. Since this network layer has a depth of 768, the time required to complete one compu-

tation task is

$$T_{\text{compute}} = \frac{1}{f} \cdot \frac{768}{h}. \quad (2)$$

The time values in these expressions are in microseconds when clock frequencies are expressed in MHz.

The estimated BRAM usage of the computing unit, in bytes, is

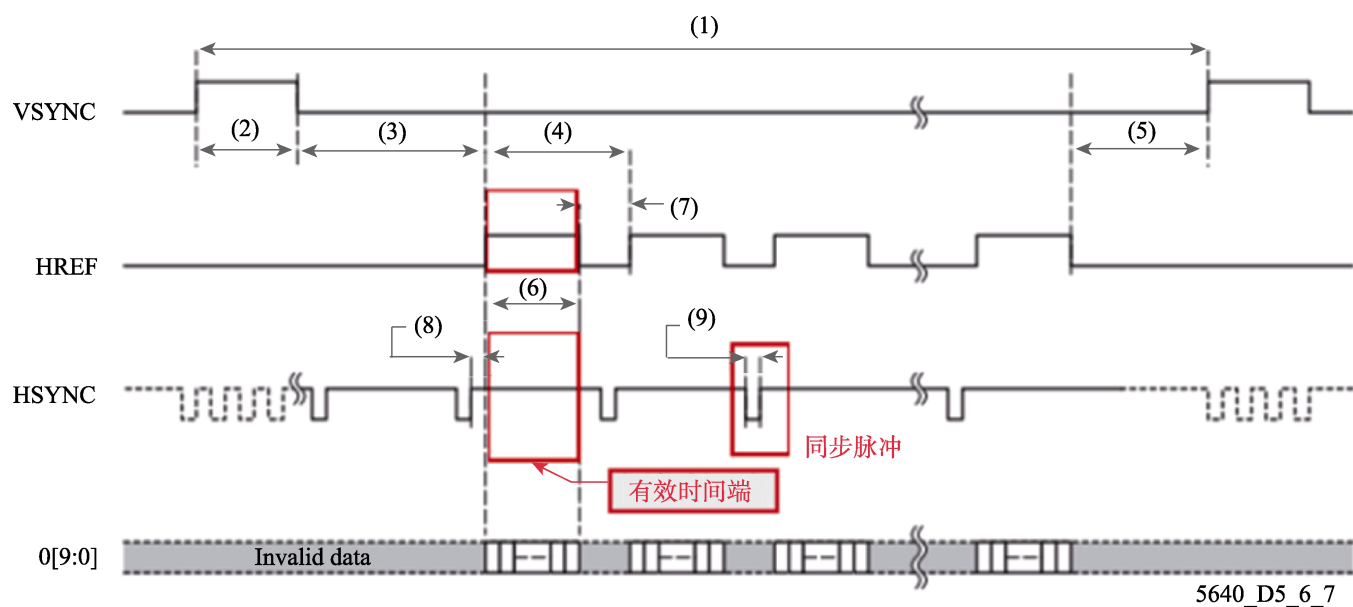
$$M_{\text{conv}} = L \cdot h \cdot H \cdot 2 + M_{\text{weights}}, \quad (3)$$

where M_{weights} is the total number of convolution-weight parameters, L is the convolution-vector width, which is 16 in this model, and H is the number of input image channels, which is 3.

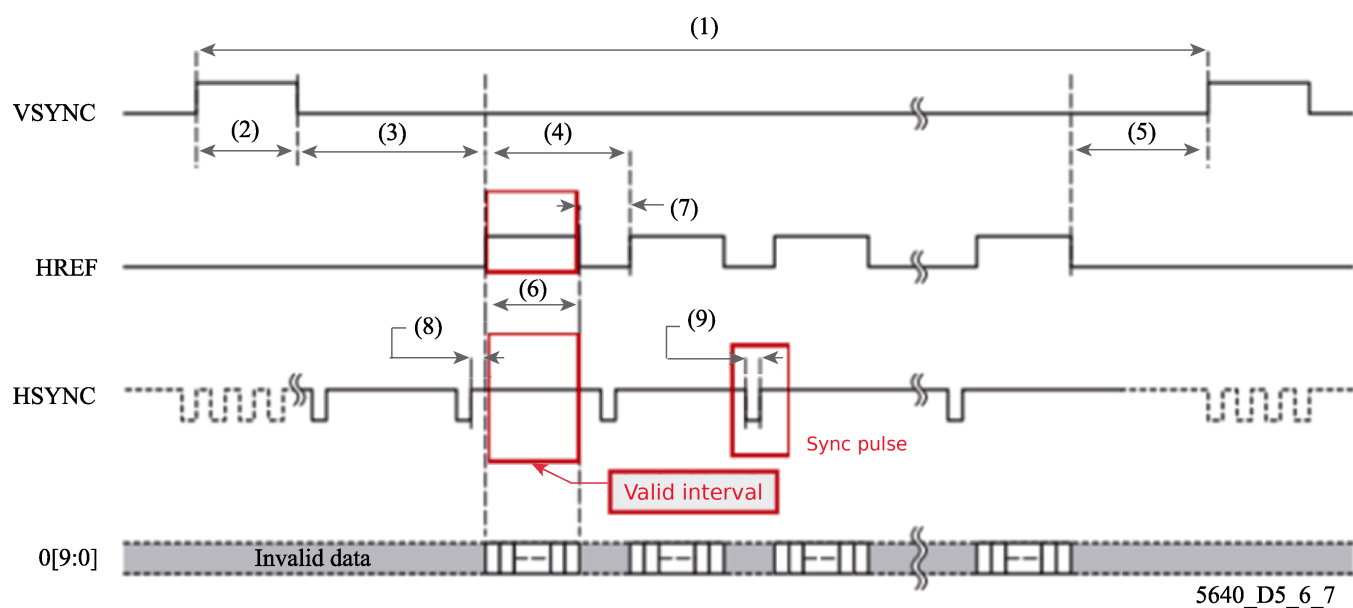
Figure 10 shows the vector convolution processing model, which includes a vector-buffer RAM unit, a computation control unit, a convolution-weight ROM unit, a vector convolution array, and a register bank. The computation control unit uses the horizontal and vertical synchronization signals and the valid signal from the normalized sensor user interface to generate addresses and enable controls for the data RAM and parameter ROM. Both address generation and valid-enable control are based on counting the valid signal. The control unit also handles clock-domain crossings for the control signals.

3.4 Accumulation Unit Design

Because the vector outputs are incomplete, the processed results must be completed through cumulative addition after convolution. Table 3 lists the remaining model parameters. After the dimension-reduced output tensor in Figure 9 has been obtained, the remaining parameters in Table 3 must be added and concatenated before subsequent network computations can proceed. To conserve

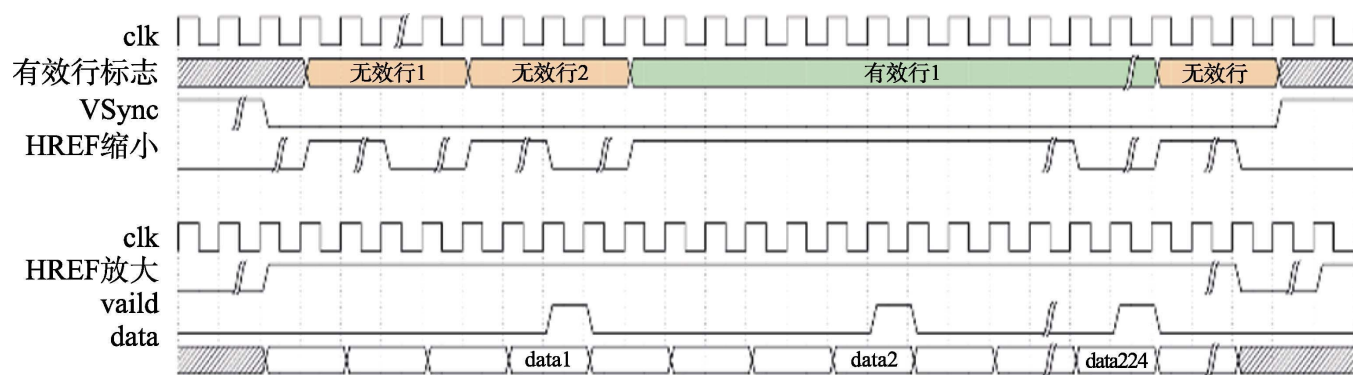


(a) Original source

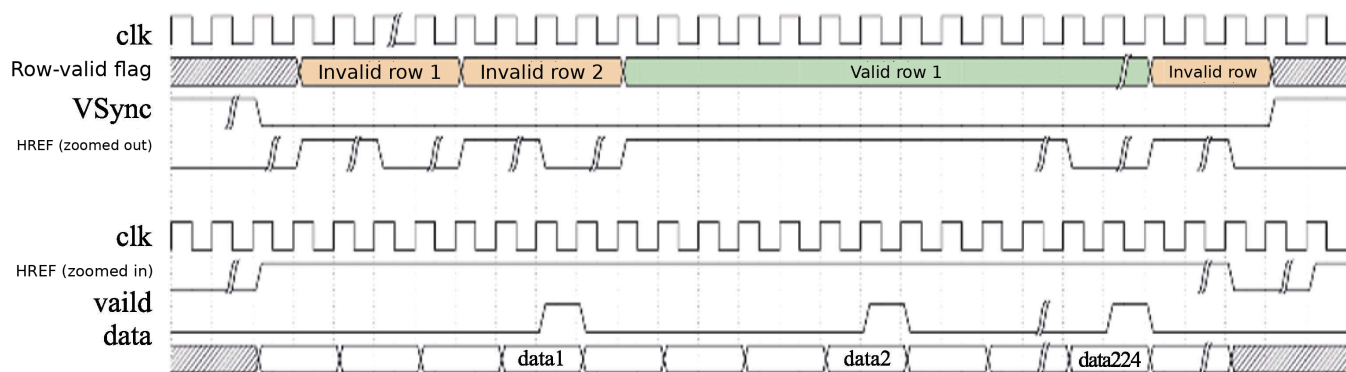


(b) English version

Figure 6: OV5640 sensor interface timing.

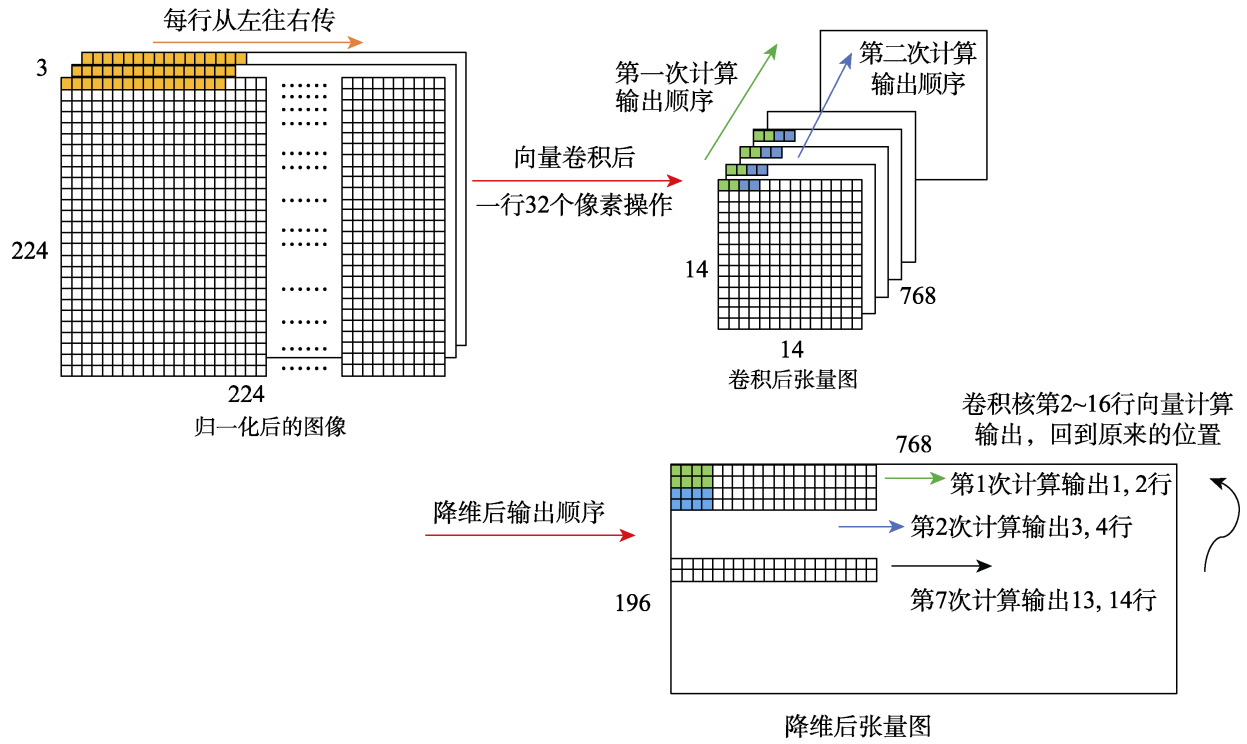


(a) Original source

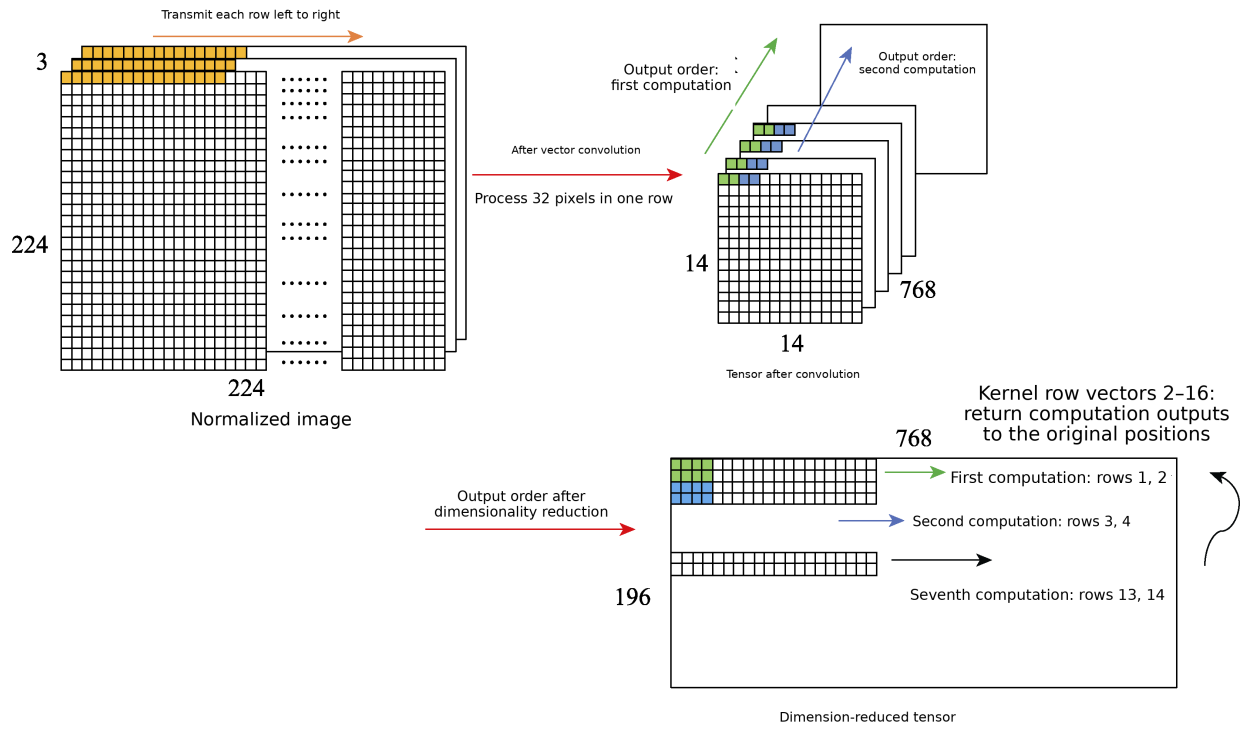


(b) English version

Figure 8: Pixel-valid timing after image preprocessing.

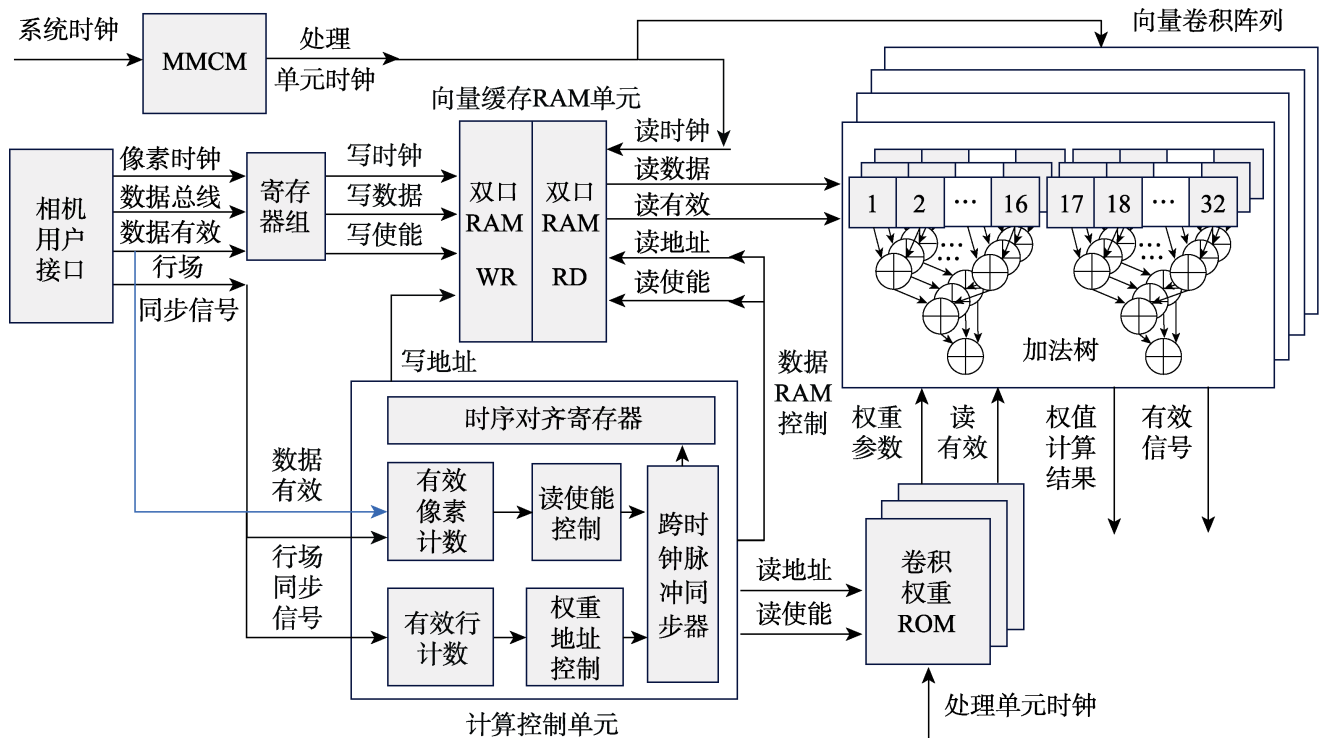


(a) Original source

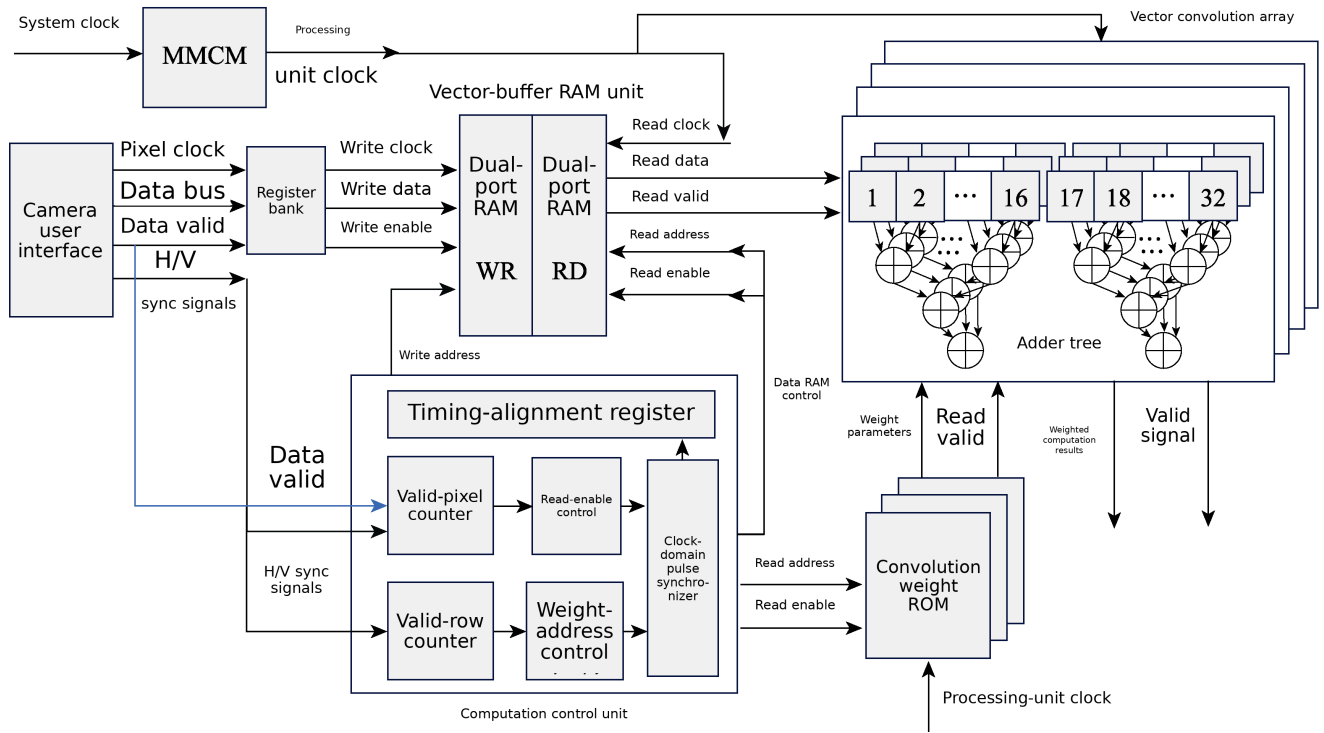


(b) English version

Figure 9: Convolution unit I/O bit-order diagram.



(a) Original source



(b) English version

Figure 10: Vector convolution processing model.

Table 3: Remaining parameters.

Parameter name	Tensor dimensions
<code>cls_token</code>	[1, 1, 768]
<code>pos_embed</code>	[1, 197, 768]
<code>patch_embed.proj.weight</code>	[768]

BRAM storage resources and reduce the amount of computation, the parameters must be combined in advance of the additions. First, the bias parameters are expanded into two dimensions by replication, and `cls_token` is concatenated at the beginning of `pos_embed`. The result is then added to the two-dimensionally expanded bias. Finally, the entire tensor is divided into a head tensor and a body tensor. Figure 11 illustrates this process.

The parameter bit width and address depth of the `.coe` file for the body tensor in Figure 11 are configured according to the output order in Figure 9. For the first convolution-kernel row vector, the configured parameters are added to the output results. For convolution-kernel row vectors 2 through 16, the preceding layer’s output is added to the previously accumulated result stored at the same address. After all vector additions have been completed, the stored values are output in single-row order and transferred to the subsequent network transmission interface. Based on these operations, the accumulation unit architecture is modeled as shown in Figure 12.

In this model, the write bandwidth is twice the read bandwidth, so the data are output only after accumulation is complete. Ignoring operation response cycles, the total time required to serialize the complete output, $T_{\text{acc_out}}$, is

$$T_{\text{acc_out}} = \frac{768}{h} \cdot 14 \cdot \frac{1}{f}. \quad (4)$$

After the current accumulated output has been completed, two new input rows must be received before the next convolution computation can begin. The minimum waiting time, T_{transfer} , is

$$T_{\text{transfer}} = 3 \times 1024 \times 2 \times \frac{1}{48}. \quad (5)$$

The associated storage resource usage is

$$M_{\text{acc_out}} = M_{\text{body}}, \quad (6)$$

where M_{body} is the size, in bytes, of the body tensor in Figure 11.

3.5 Transmission Buffer Unit Design

This design uses UDP transmission [20]. Encapsulated data pass from the UDP layer to the Internet Protocol (IP) layer, are repackaged at the media access control (MAC) layer, and are then transmitted over Gigabit Ethernet. The UDP protocol stack’s user interface transfers one byte at a time. In the computational output,

the integer and fractional parts each occupy 1 B (byte). Because a UDP packet can carry at most 1472 B per transmission, and the dimension-reduced output tensor in Figure 9 gives 768 B of vector computation data per row, a complete row of computational data cannot be output in a single transmission.

To preserve the integrity of each UDP-transmitted data value, maintain consistent fragment lengths, and ensure that the subsequent processing unit can identify a vector’s position within the tensor, the high-level pulses of the vertical synchronization signal and the input-valid signal are counted to determine the tensor row number. Each row is transmitted in two parts. A flag byte indicates whether the data belong to the first or second half of the row. The flag byte and the row-number byte are concatenated in that order and inserted before the valid segment of the accumulated output data. The resulting concatenated data are then transmitted as a UDP packet. Because the UDP user clock and convolution processing clock have different frequencies, and because the read/write clocks and data widths also differ, the storage unit must be connected as a first-in, first-out (FIFO) buffer before the UDP protocol stack input. Figure 13 shows the transmission buffer unit.

In this model, FIFO reads are slower than writes. When written data are present, the output data are buffered through the FIFO. Since the protocol stack clock is 125 MHz and the FIFO serializes the output one byte at a time, the data transmission interval T_{send} is

$$T_{\text{send}} = 768 \times 2 \times 14 \times \frac{1}{125}. \quad (7)$$

Let $T_{\text{udp_ready}}$ denote the time interval required for the next transmission. Its value is

$$T_{\text{udp_ready}} = 16 \times 1024 \times 2 \times \frac{1}{48}. \quad (8)$$

The associated storage resource usage is

$$M_{\text{send}} = M_{\text{body}}. \quad (9)$$

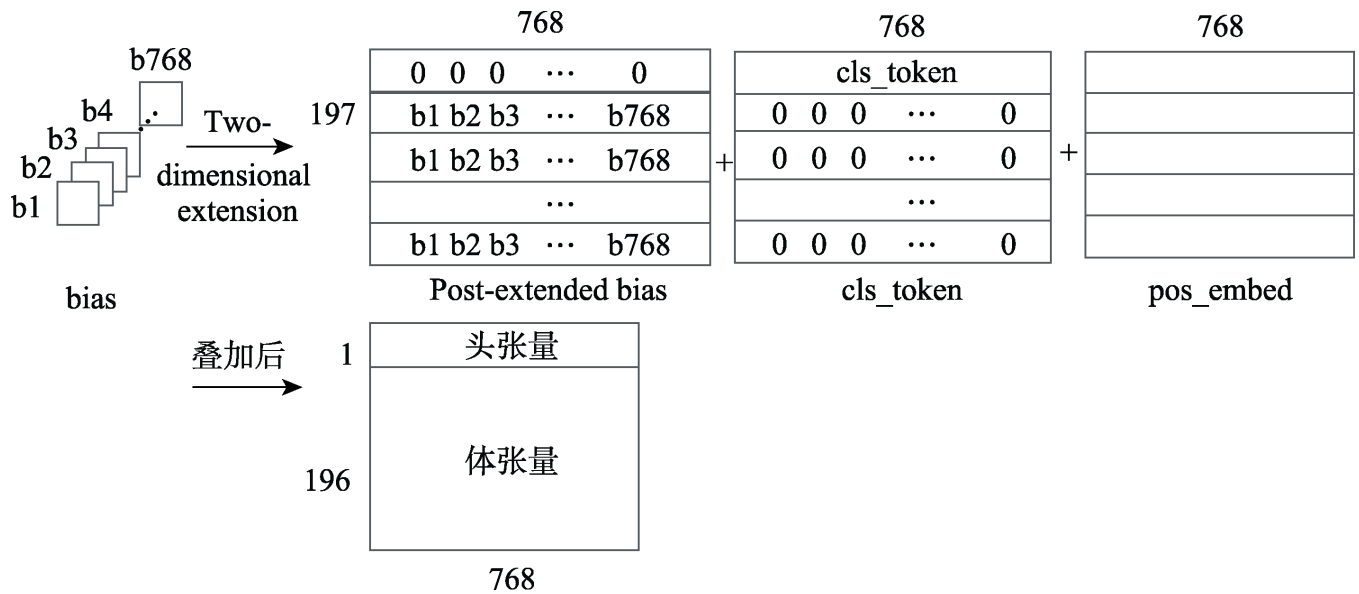
3.6 Resource and Clock Evaluation and Device Selection

Based on the system partitioning and modeling process, combining Equations (1)–(9) gives

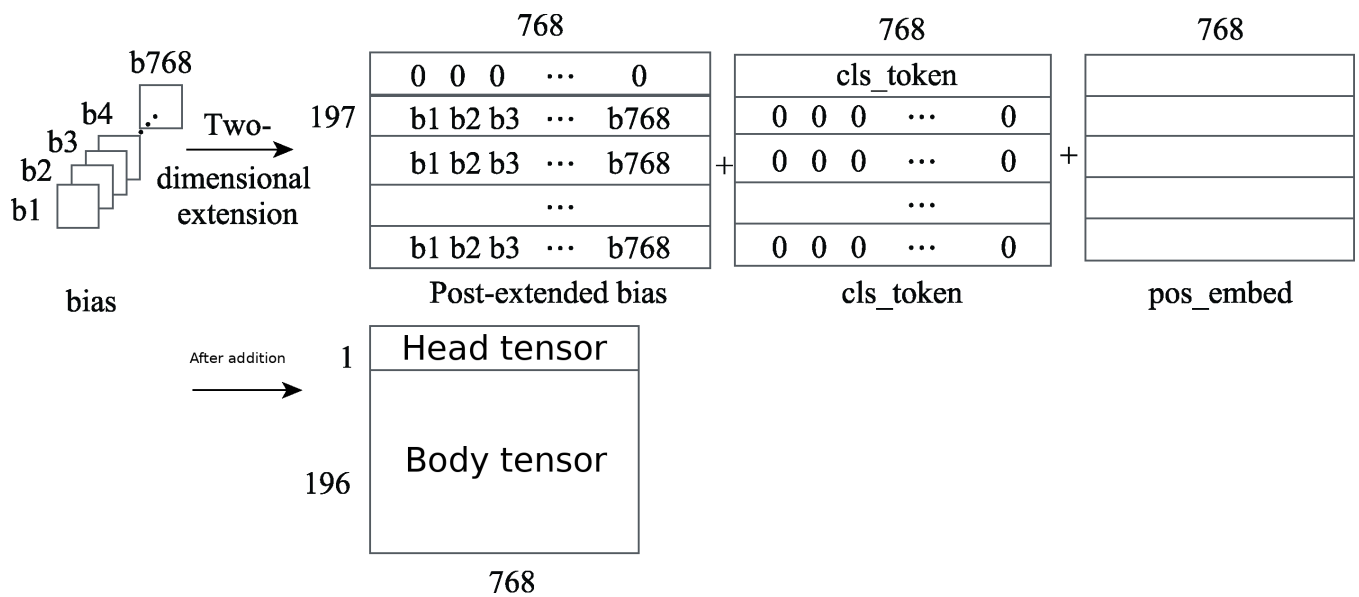
$$\begin{cases} T_{\text{buffer}} > T_{\text{compute}}, \\ T_{\text{acc_out}} < T_{\text{transfer}}, \\ T_{\text{send}} < T_{\text{udp_ready}}, \\ M_{\text{total}} = M_{\text{conv}} + M_{\text{acc_out}} + M_{\text{send}}. \end{cases} \quad (10)$$

Solving these constraints yields

$$\begin{cases} h \cdot f > 144, \\ M_{\text{total}} = 96 \cdot h + 497\,664. \end{cases} \quad (11)$$



(a) Original source



(b) English version

Figure 11: Bias parameter processing procedure.

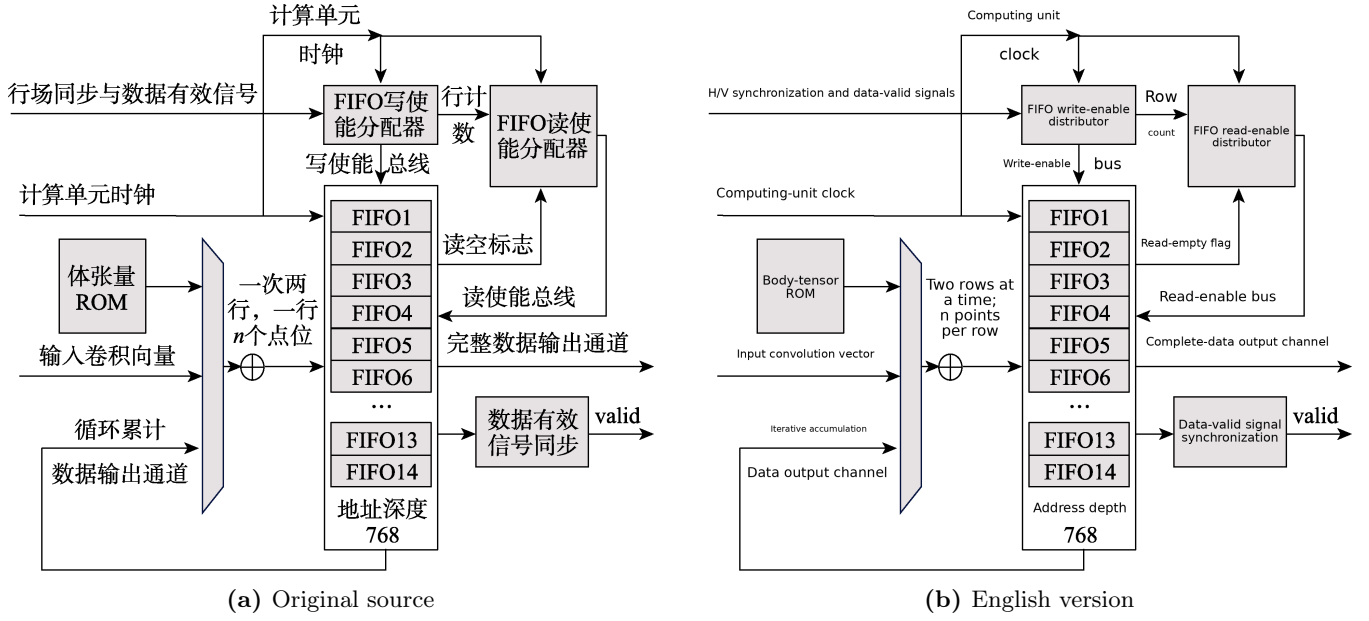


Figure 12: Accumulation unit architecture model. FIFO denotes first-in, first-out, a queue-processing discipline.

The computational processing unit model therefore admits a solution for the degree of parallelism and the computing clock frequency. For the FPGA system, the minimum value of h is 1, and the clock frequency f is required to be at least 144 MHz. A frequency of 144 MHz is well below the physical clock-frequency limit of the FPGA technology. This design operates the computing unit at 250 MHz and sets $n = 4$.

To ensure that sufficient logic resources remain available for other tasks, this design uses a ZYNQ7045 device to manage the resources. Figure 14 shows the utilization of the device’s internal resources after the design is complete. The numerical data from that figure are also reproduced in tabular form.

4 Simulation Verification and Top-Level Optimization

4.1 Simulation Verification

First, a Python program separates the $1024 \text{ px} \times 768 \text{ px}$ image data into three channels in RGB565 format and saves each channel to a separate text file in binary form. Each file contains 768 rows of data. Each pixel occupies 5, 6, or 5 bits in the respective channel, and each row contains data for 1024 pixels. Adjacent pixel values are concatenated directly, without delimiters.

Next, a testbench model is constructed to simulate the interface timing inputs according to the interface timing specified in the OV5640 technical manual. To verify the model’s computational results, the text files storing the model are initialized. Whenever the output results are valid, the output data are stored in sequence. Simulation stops once the computational results for an entire

image frame have been output. The results are then passed to a Python program, which reads the text file and converts the data into floating-point `torch` format. The head tensor is prepended to the data, and the subsequent computations are performed.

Figure 15 compares the PyTorch-based compilation and verification results with those obtained from Verilog compilation and verification. Because the original model was not converted to operate at $1024 \text{ px} \times 768 \text{ px}$, some error is present in the computed results.

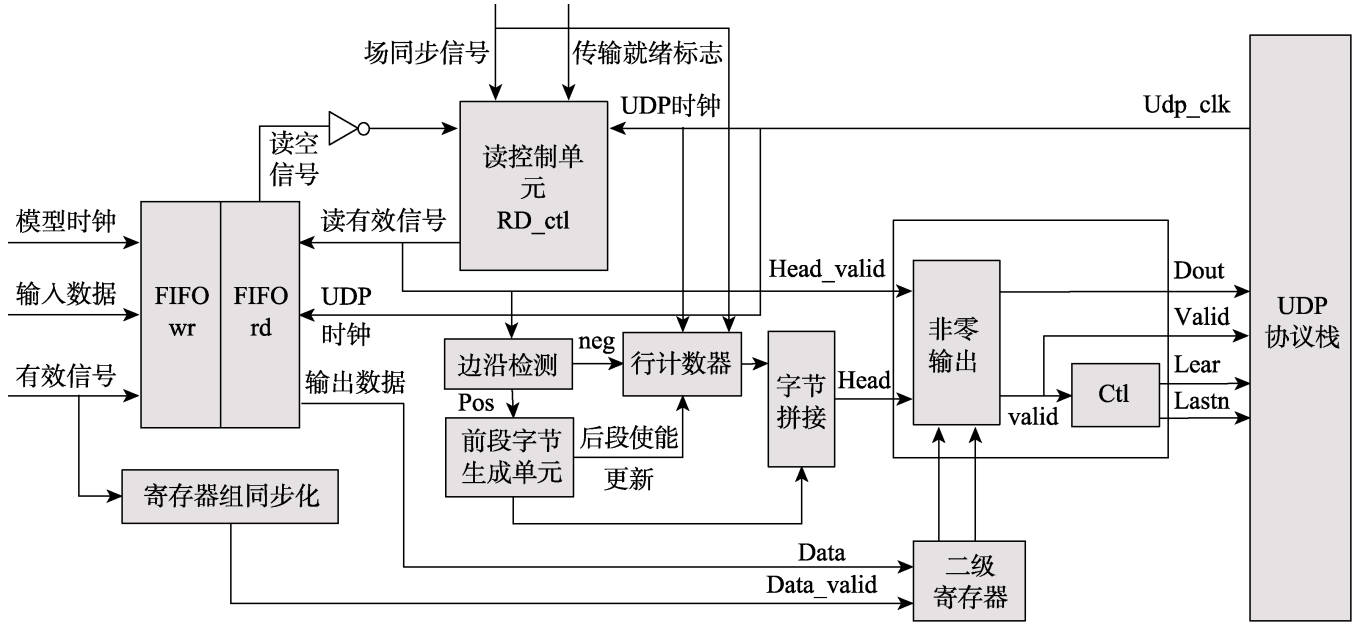
4.2 Timing and Resource Optimization

Figure 16 shows the timing violation report generated during placement and routing. The report indicates that the data paths do not meet the setup-time requirements.

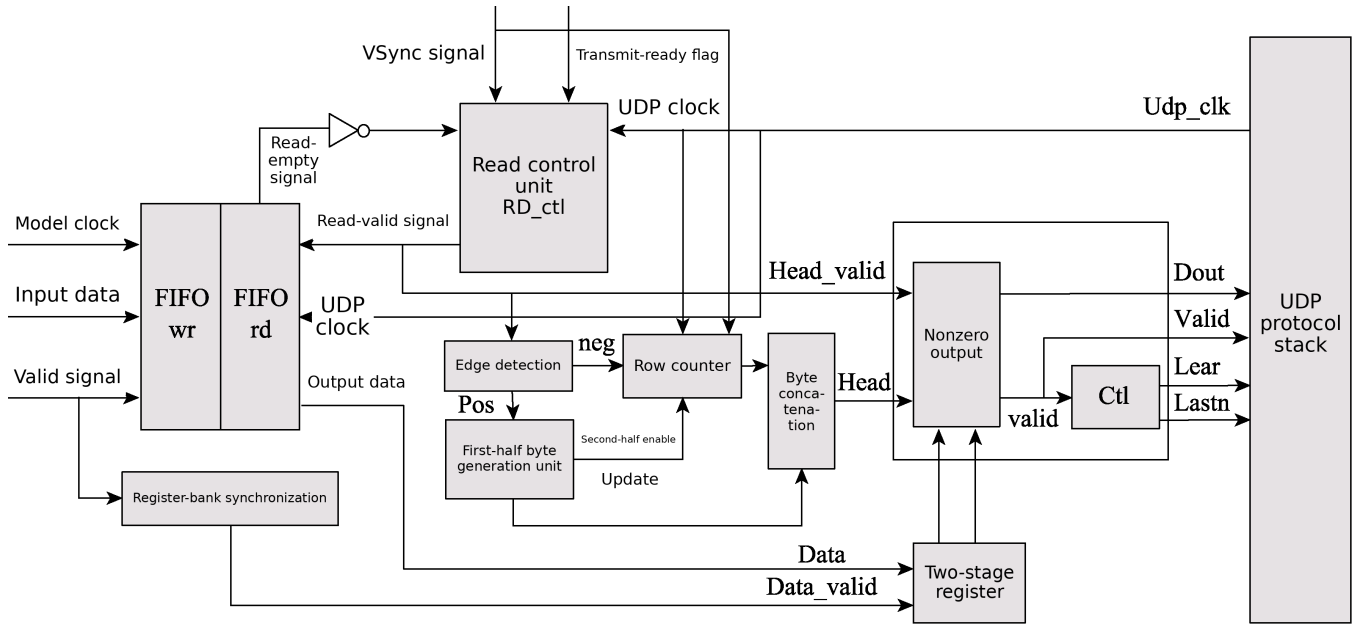
Analysis of the electronic design automation (EDA) reports identifies two types of design problems. The first type involves timing problems caused by excessive logic depth in the multiplier chain, as shown in Figure 17. This problem can be resolved by increasing the pipeline depth from one stage to three stages.

The second type of design problem is the concentration of BRAM resources used for weight parameters, which causes routing congestion and timing violations. To address this problem, the partitioning configuration for the smallest ROM units must be selected so that its configuration size is closest to the least common multiple of the configuration size and the convolution-vector size. Figure 18 shows the source of the routing congestion.

After these two types of problems have been corrected, the placement-and-routing results are as shown in Figure 19. For setup timing, the worst negative slack (WNS) is 0.128 ns; for hold timing, the worst hold slack (WHS)



(a) Original source



(b) English version

Figure 13: Transmission buffer unit.

Resource	Utilization	Available	Utilization %
LUT	46433	218600	21.24
LUTRAM	1010	70400	1.43
FF	45535	437200	10.42
BRAM	307.50	545	56.42
DSP	2	900	0.22
IO	113	362	31.22
BUFG	16	32	50.00
MMCM	5	8	62.50
PLL	1	8	12.50

(a) Original source

Resource	Utilization	Available	Utilization %
LUT	46433	218600	21.24
LUTRAM	1010	70400	1.43
FF	45535	437200	10.42
BRAM	307.50	545	56.42
DSP	2	900	0.22
IO	113	362	31.22
BUFG	16	32	50.00
MMCM	5	8	62.50
PLL	1	8	12.50

(b) English version

Figure 14: Utilization of the device's internal resources.

```

运行: predict x
[-1.8047, -3.0430],
[-1.9531, -2.3086,
-1.9102, ..., -2.3867,
-2.0664, -2.9648]])
torch.Size([197, 768])
class: car      prob: 0.181
class: people   prob: 0.819

```

```

predict (1) x
D:\anaconda\envs\sdsd_torch\pytho
.py
class: car      prob: 0.185
class: people   prob: 0.815

```

(a) 基于pytorch的编译验证结果

(b) Verilog编译验证结果

(a) Original source

```

Run: predict x
[-1.8047, -3.0430],
[-1.9531, -2.3086,
-1.9102, ..., -2.3867,
-2.0664, -2.9648]])
torch.Size([197, 768])
class: car      prob: 0.181
class: people   prob: 0.819

```

```

predict (1) x
D:\anaconda\envs\sdsd_torch\pytho
.py
class: car      prob: 0.185
class: people   prob: 0.815

```

(a) PyTorch compilation and verification results

(b) Verilog compilation and verification results

(b) English version

Figure 15: Comparison of verification results. Source panels: (a) PyTorch-based compilation and verification results; (b) Verilog compilation and verification results.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -0.513 ns	Worst Hold Slack (WHS): 0.044 ns	Worst Pulse Width Slack (WPWS): 1.100 ns
Total Negative Slack (TNS): -501.096 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 3438	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 39368	Total Number of Endpoints: 39269	Total Number of Endpoints: 14822

Timing constraints are not met.

(a) Original source

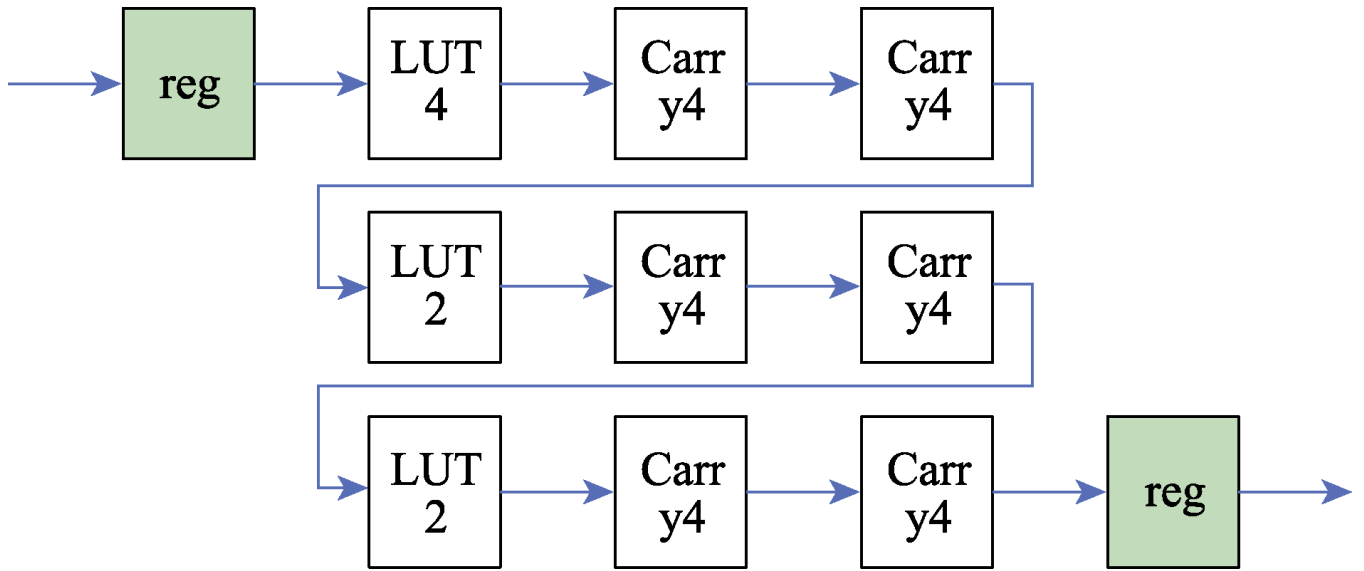
Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -0.513 ns	Worst Hold Slack (WHS): 0.044 ns	Worst Pulse Width Slack (WPWS): 1.100 ns
Total Negative Slack (TNS): -501.096 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 3438	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 39368	Total Number of Endpoints: 39269	Total Number of Endpoints: 14822

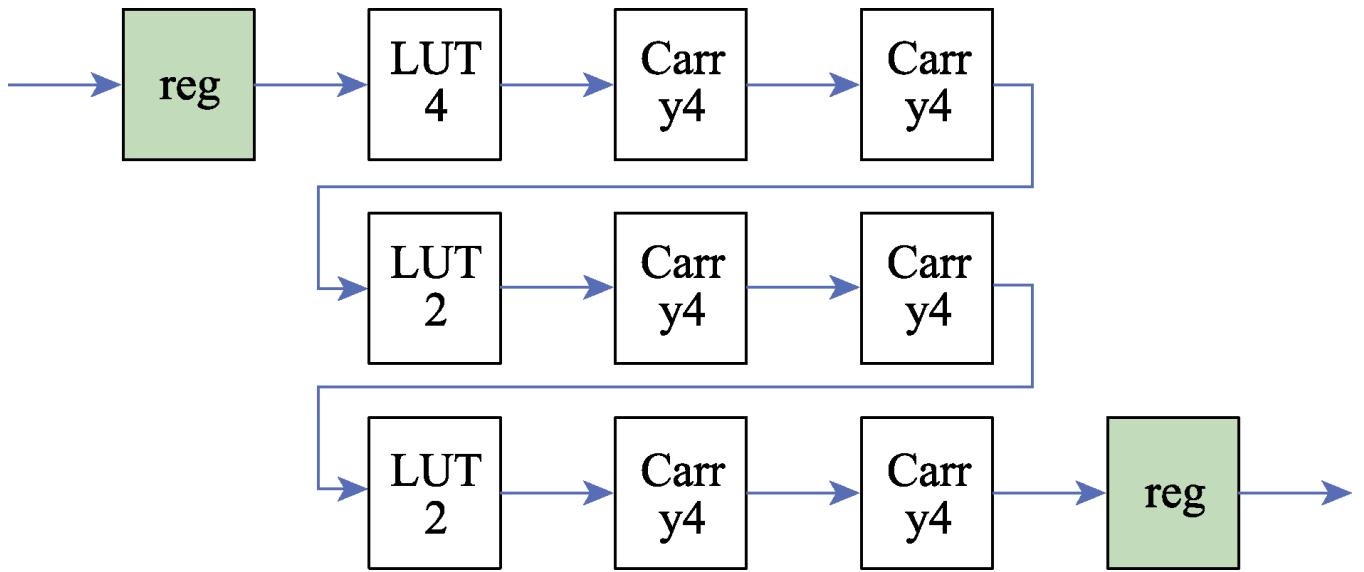
Timing constraints are not met.

(b) English version

Figure 16: Timing violation report from placement and routing.

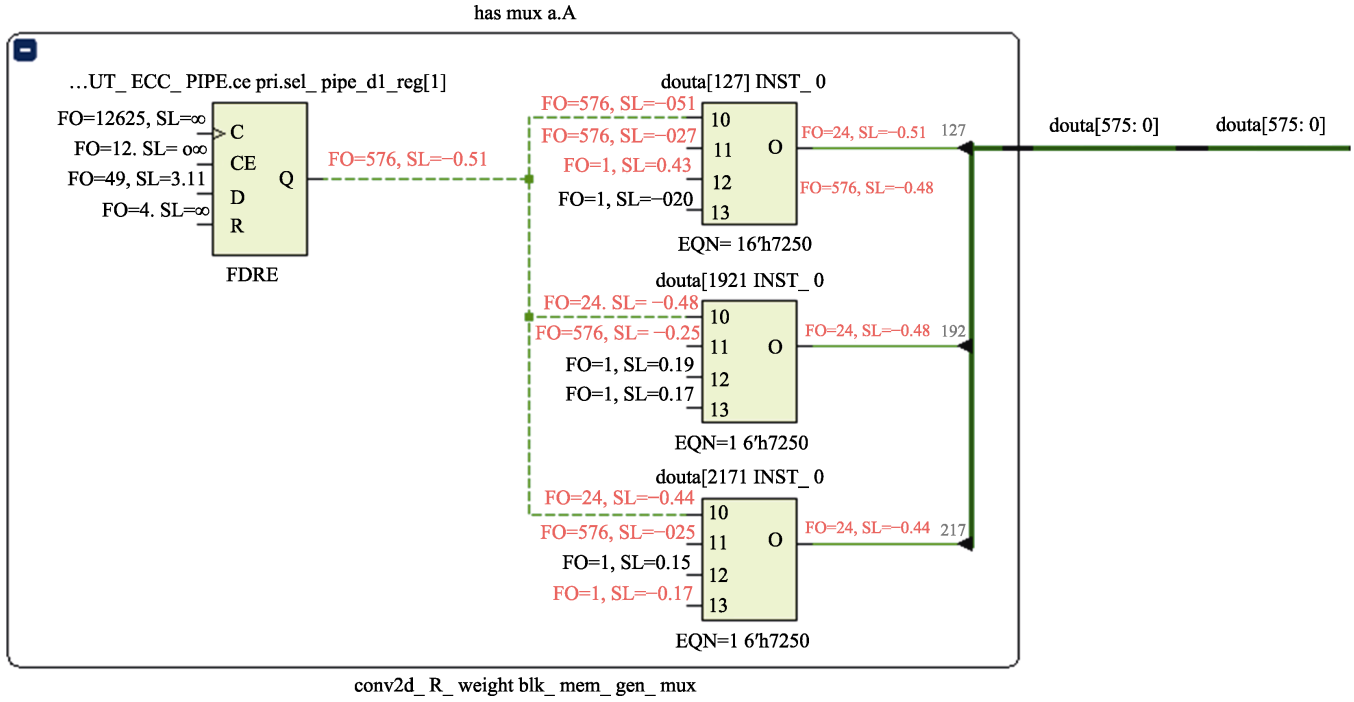


(a) Original source

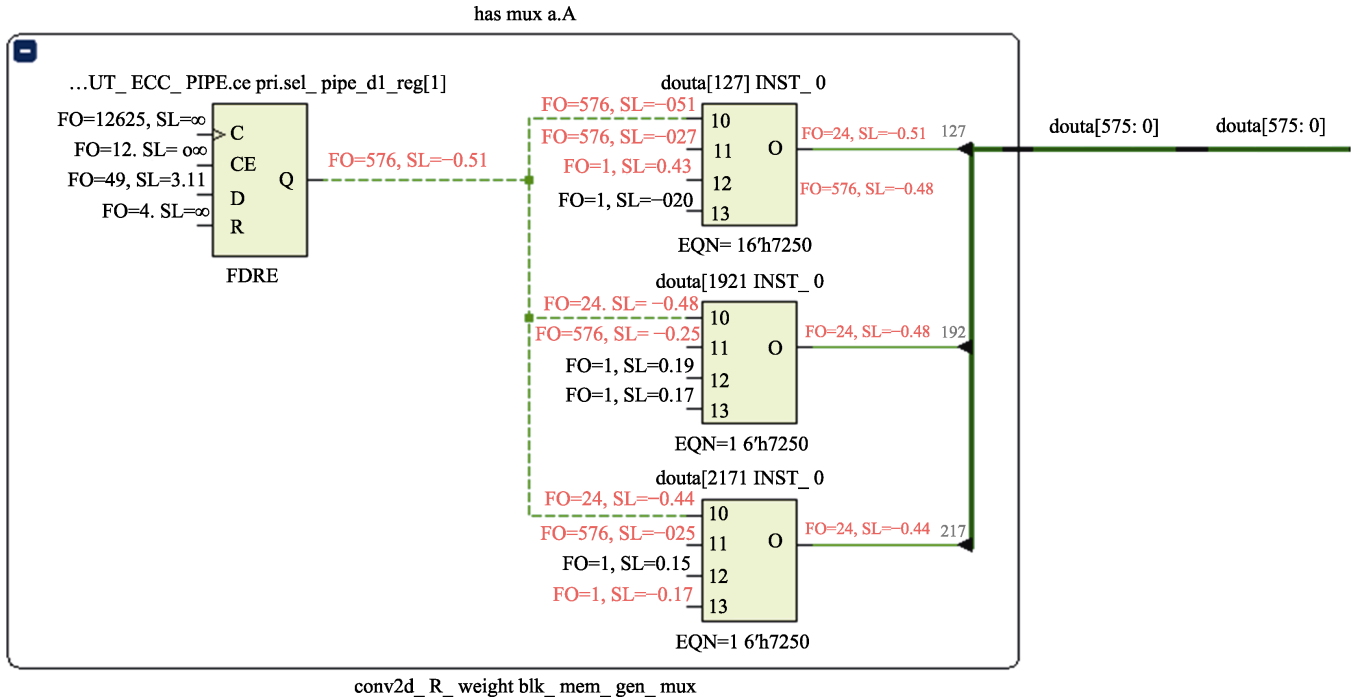


(b) English version

Figure 17: Multiplier-chain problem.



(a) Original source



(b) English version

Figure 18: Source of routing congestion.

Resource utilization data from Figure 14.

Resource	Utilization	Available	Utilization (%)
LUT	46,433	218,600	21.24
LUTRAM	1,010	70,400	1.43
FF	45,535	437,200	10.42
BRAM	307.50	545	56.42
DSP	2	900	0.22
IO	113	362	31.22
BUFG	16	32	50.00
MMCM	5	8	62.50
PLL	1	8	12.50

is 0.048 ns. Both values are greater than zero, so the design meets the timing requirements.

Figure 20 shows the placement-and-routing layouts before and after optimization. The resource placement shows that the overall resources are arranged from left to right and from top to bottom, satisfying the sequential placement requirements of the I/O interfaces.

5 Multithreaded Design and Execution Results

To facilitate the use of the computational results processed and transmitted by the FPGA, the implementation is first run on a PC and then migrated to the NPU for further development. First, the deep portion of the original model is extracted, and all code using `torch` library functions is consolidated into a model file named `deep_model.py`. The head tensor described in Figure 11 is then placed at the beginning of the list that stores subsequent input data. Finally, the UDP protocol stack is initialized, the local IP address and port number are bound, and data reception begins.

The first thread is created to receive UDP packets transmitted by the FPGA and place them in a queue. Starting this thread enables UDP reception. The received hexadecimal data are converted to binary, and the first and second bytes are extracted and interpreted as the first-/second-half flag and the row number, respectively. Queue storage begins when the queue is empty and both the data row number and the first byte are zero. Storage into the queue stops when both the data row number and the first byte reach their preset maximum values. This operation prevents cumulative latency in queue storage even when writes are faster than reads, while ensuring the integrity of the stored frame data.

After thread 1 starts, the program enters its main loop. In each iteration, the main thread first reads a data segment from the completed queue. It then creates and starts threads 2 and 3. Thread 2 processes the binary data in the first half of the segment, including the integer and fractional parts, converts the data to decimal format, and stores them at its assigned quarter-position

in the first half, corresponding to data bytes 1–512 of the complete row. Thread 3 processes the binary data in the second half of the segment, including the integer and fractional parts, converts the data to decimal format, and stores them at its assigned quarter-position in the second half, corresponding to data bytes 513–1024 of the complete row.

Once threads 2 and 3 have completed their respective tasks, the main thread reads the first and second bytes of the current segment header, which contain control information, and uses this information to initiate the next round of data conversion and storage. At the same time, the main thread checks whether the current segment is the final fragment of the row, that is, whether the row number is at its maximum and the segment flag indicates the final segment. If this condition is met, the model’s deep-network computation task is started and the NPU is invoked to perform the computation. Figure 21 shows the thread-processing program flow.

Through multithreading and thread-pool processing, image-form recognition results are converted into NPU execution results, as shown in Figure 22. The mean recognition accuracies for people and vehicles are 85% and 90.5%, respectively. As shown in Figure 23, the model’s average execution time per frame is reduced from an initial 0.6439 s to 0.0995 s.

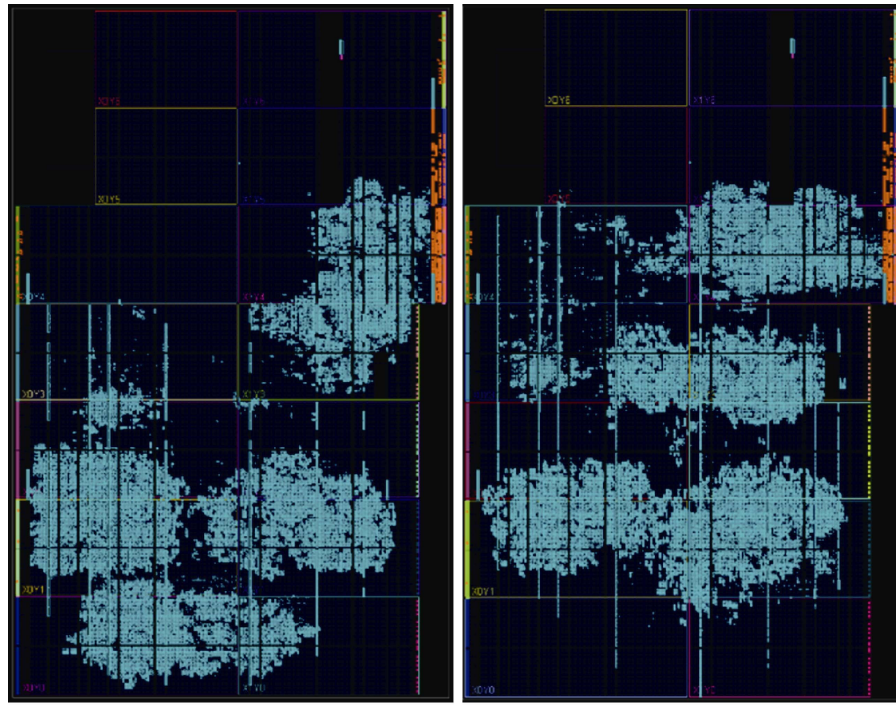
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.128 ns	Worst Hold Slack (WHS): 0.048 ns	Worst Pulse Width Slack (WPWS): 1.100 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 83610	Total Number of Endpoints: 83511	Total Number of Endpoints: 38692
All user specified timing constraints are met.		

(a) Original source

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.128 ns	Worst Hold Slack (WHS): 0.048 ns	Worst Pulse Width Slack (WPWS): 1.100 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 83610	Total Number of Endpoints: 83511	Total Number of Endpoints: 38692
All user specified timing constraints are met.		

(b) English version

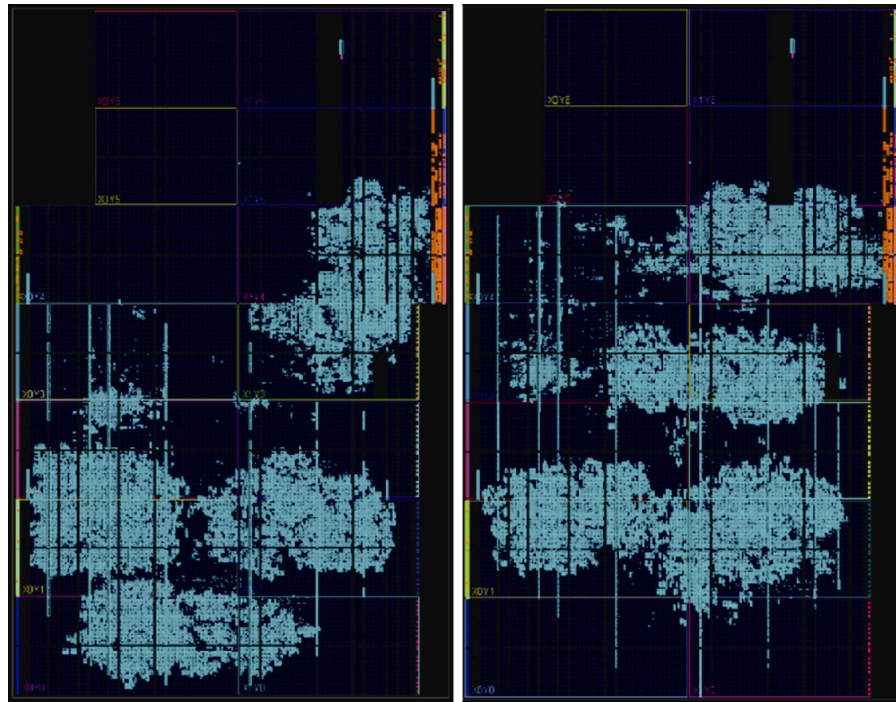
Figure 19: Timing report after optimization.



(a) 优化前的布局布线图

(b) 优化后的布局布线图

(a) Original source

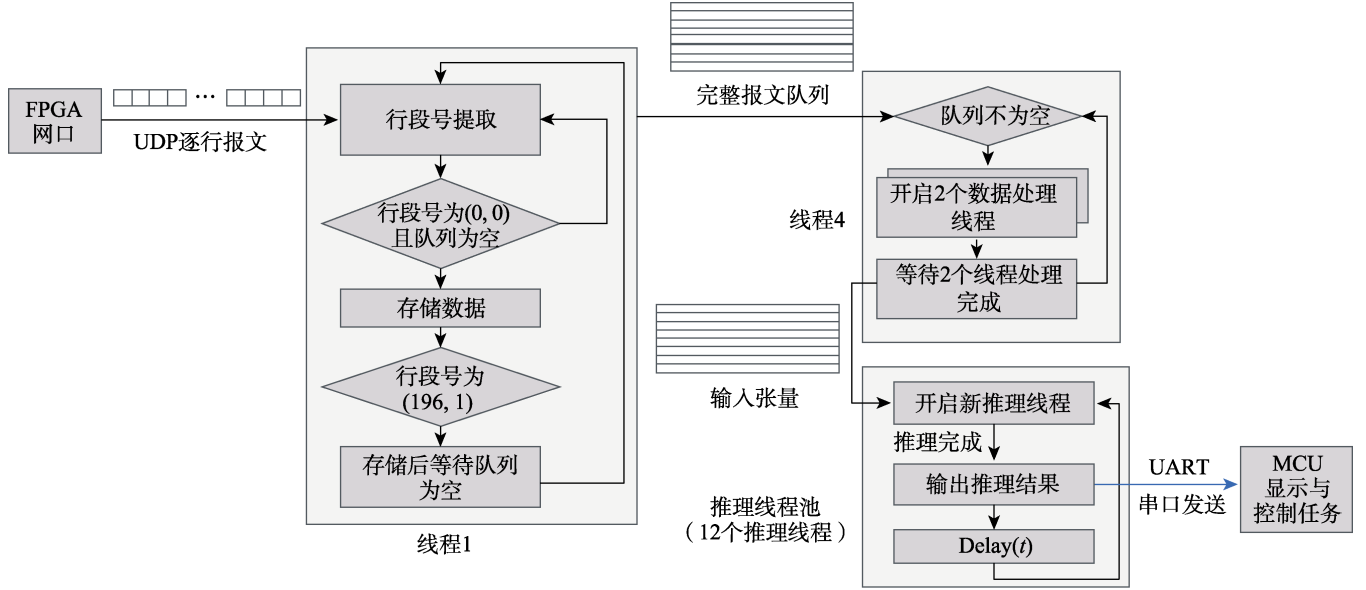


(a) Placement and routing before optimization

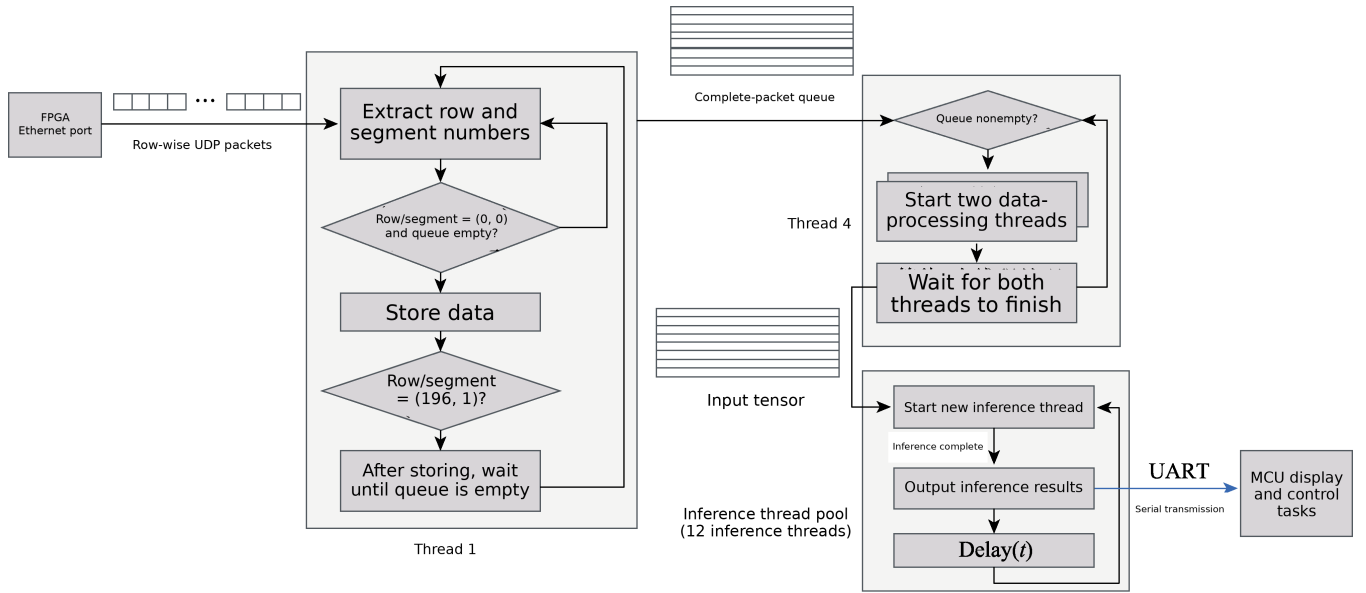
(b) Placement and routing after optimization

(b) English version

Figure 20: Placement-and-routing layouts. Source panels: (a) before optimization; (b) after optimization.

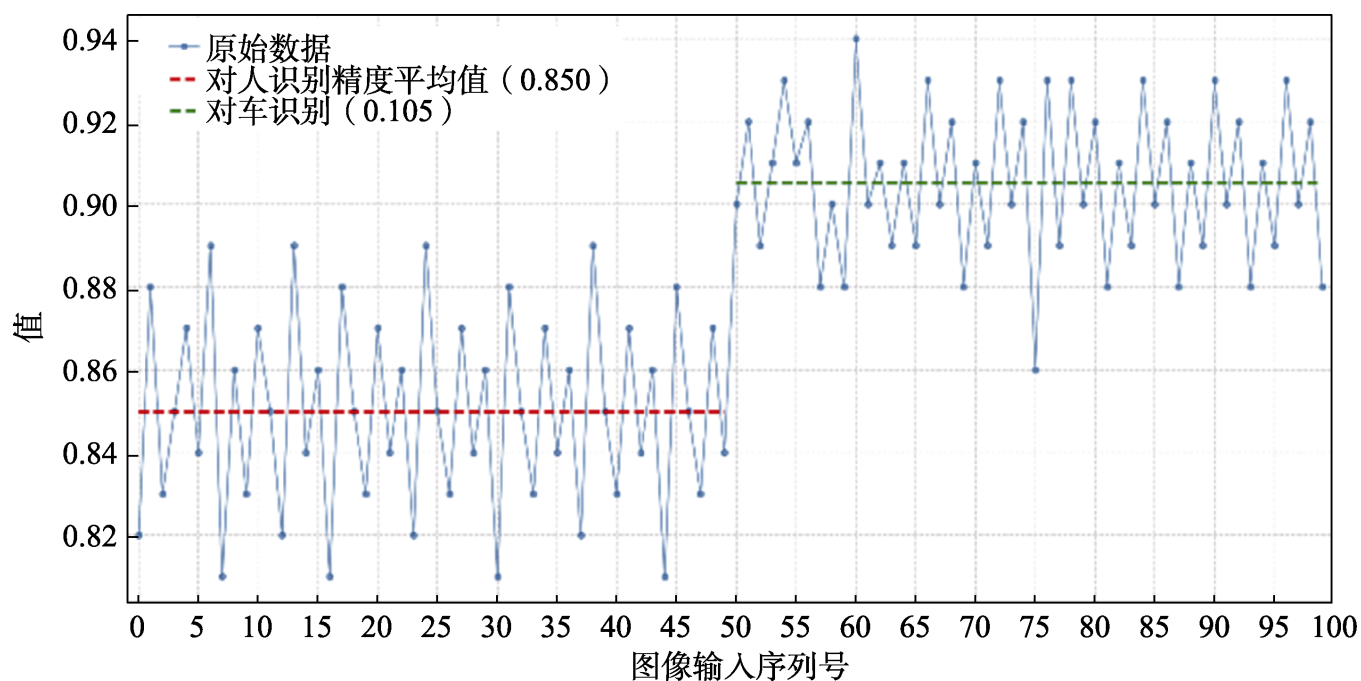


(a) Original source

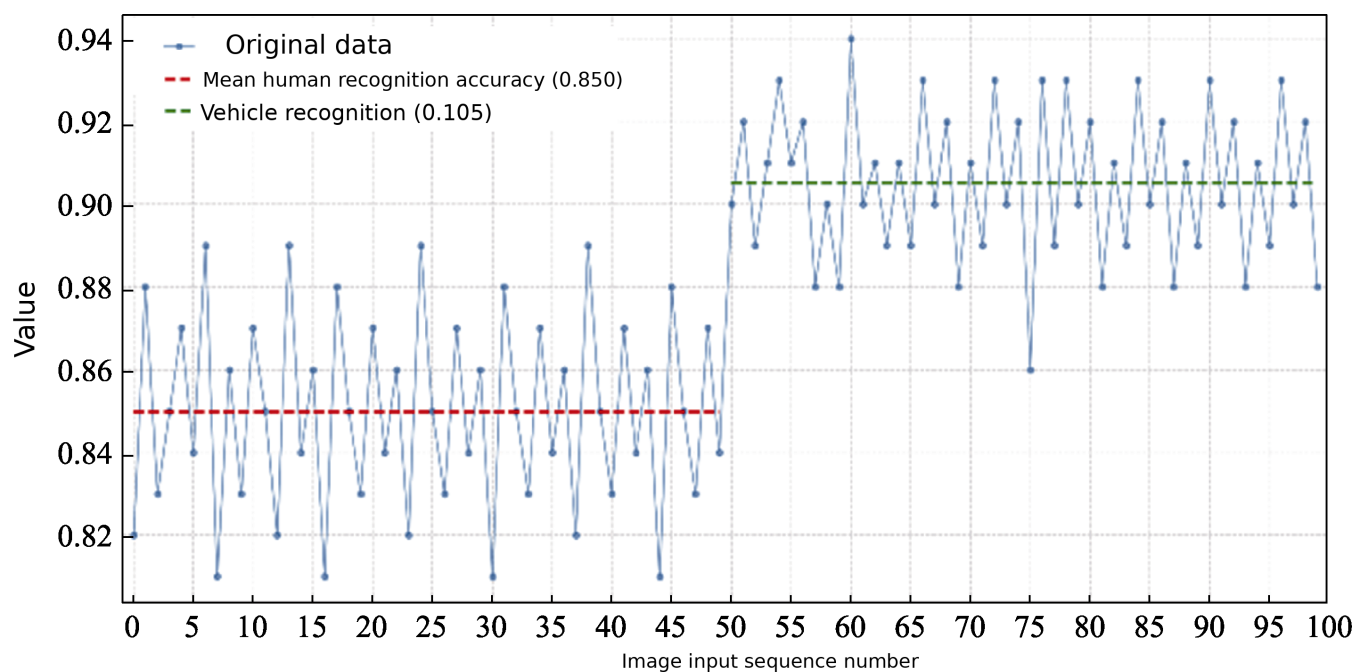


(b) English version

Figure 21: Thread-processing program flow. Delay(t) denotes a delay, where t is the algorithm inference time in a single-threaded configuration divided by the total number of threads in the multithreaded configuration.

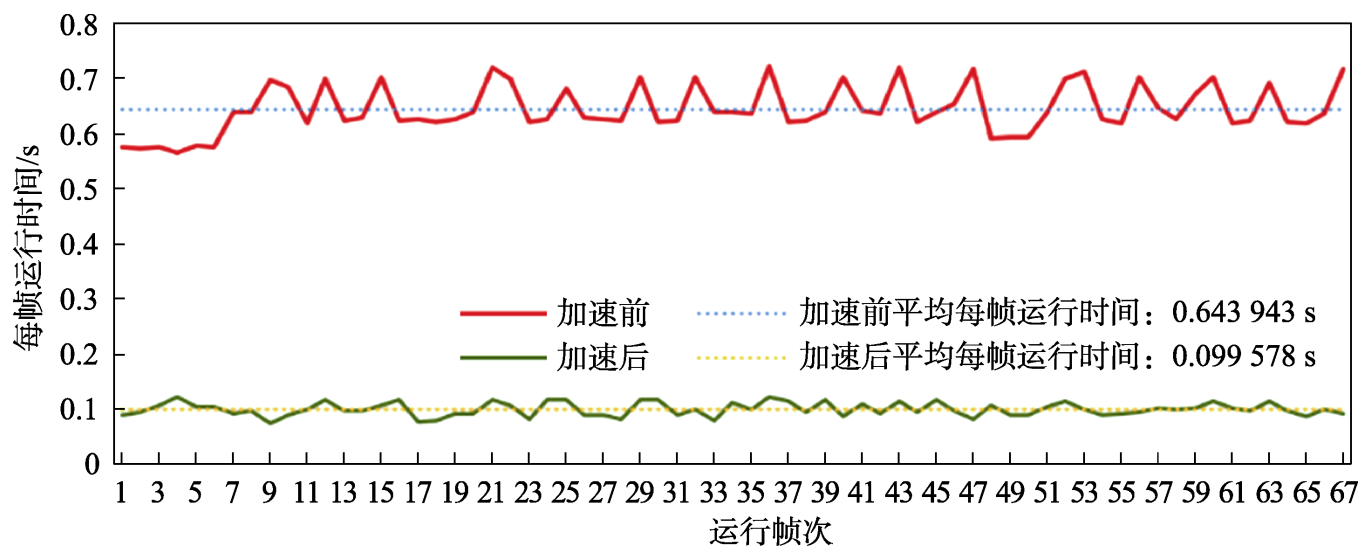


(a) Original source

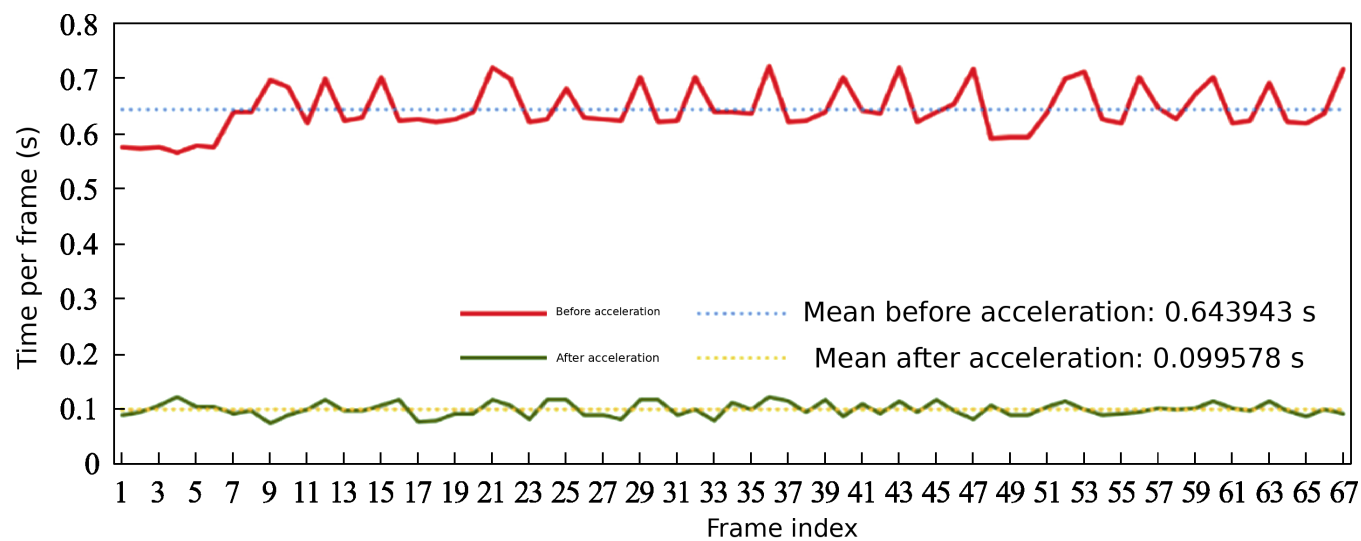


(b) English version

Figure 22: Classification results.



(a) Original source



(b) English version

Figure 23: Execution time per frame.

References

- [1] DOSOVITSKIY A, BEYER L, KOLESNIKOV A, et al. An image is worth 16x16 words: Transformers for image recognition at scale [Z/OL]. (2020-10-22) [2025-04-08]. <https://arxiv.org/pdf/2010.11929>.
- [2] HAN K, WANG Y H, CHEN H T, et al. A survey on vision transformer [J]. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022, 45(1): 87–110.
- [3] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need [J]. *Advances in Neural Information Processing Systems*, 2017, 30.
- [4] LIU Z, LIN Y, CAO Y, et al. Swin transformer: Hierarchical vision transformer using shifted windows [C]//*Proceedings of the IEEE/CVF International Conference on Computer Vision*. New York, NY: Institute of Electrical and Electronics Engineers, 2021: 10012–10022.
- [5] TOUVRON H, CORD M, DOUZE M, et al. Training data-efficient image transformers & distillation through attention [C]//*International Conference on Machine Learning*. New York: *Proceedings of Machine Learning Research*, 2021: 10347–10357.
- [6] KHAN S, NASEER M, HAYAT M, et al. Transformers in vision: A survey [J]. *ACM Computing Surveys (CSUR)*, 2022, 54(10s): 1–41.
- [7] SZE V, CHEN Y H, YANG T J, et al. Efficient processing of deep neural networks: A tutorial and survey [J]. *Proceedings of the IEEE*, 2017, 105(12): 2295–2329.
- [8] NURVITADHI E, VENKATESH G, SIM J, et al. Can FPGAs beat GPUs in accelerating next-generation deep neural networks? [C]//*Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. New York: Association for Computing Machinery, 2017: 5–14.
- [9] ZHANG C, LI P, SUN G Y, et al. Optimizing FPGA-based accelerator design for deep convolutional neural networks [C]//*Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. New York: Association for Computing Machinery, 2015: 161–170.
- [10] ZHAO R, NIU X, WU Y, et al. Optimizing CNN-based object detection algorithms on embedded FPGA platforms [C]//*2017 IEEE International Symposium on Circuits and Systems (ISCAS)*. New York: Institute of Electrical and Electronics Engineers, 2017: 1–4.
- [11] HOWARD A G, ZHU M, CHEN B, et al. MobileNets: Efficient convolutional neural networks for mobile vision applications [Z/OL]. (2017-04-17) [2025-04-08]. <https://arxiv.org/pdf/1704.04861>.
- [12] SHEN J, HUANG Y, WANG Z, et al. Towards a uniform template-based architecture for accelerating 2D and 3D CNNs on FPGA [C]//*Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. New York: Association for Computing Machinery, 2018: 97–106.
- [13] CHEN Y H, KRISHNA T, EMER J S, et al. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks [J]. *IEEE Journal of Solid-State Circuits*, 2016, 52(1): 127–138.
- [14] ALWANI M, CHEN H, FERDMAN M, et al. Fused-layer CNN accelerators [C]//*Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture*. New York: Institute of Electrical and Electronics Engineers, 2016: 1–12.
- [15] MA Y F, CAO Y, VRUDHULA S, et al. Optimizing loop operation and dataflow in FPGA acceleration of deep convolutional neural networks [C]//*Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. New York: Association for Computing Machinery, 2017: 45–54.
- [16] DING W, HUANG Z Y, HUANG Z K, et al. Designing efficient accelerator of depthwise separable convolutional neural network on FPGA [J]. *Journal of Systems Architecture*, 2019, 97: 278–286.
- [17] CHEN Y H, KRISHNA T, EMER J S, et al. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks [J]. *IEEE Journal of Solid-State Circuits*, 2016, 52(1): 127–138.
- [18] LI E, ZENG L, ZHOU X, et al. Edge AI: On-demand accelerating deep neural network inference via edge computing [J]. *IEEE Transactions on Wireless Communications*, 2020, 19(1): 447–457.
- [19] POSTEL J. User datagram protocol: RFC 768 [R]. Los Angeles, CA: Institute for Scientific Information, 1980.

[20] OmniVision Technologies. OV5640 1/4-inch CMOS UXGA camera chip sensor datasheet [Z]. Rev. 1.3–ed. Santa Clara: OmniVision Technologies, 2011.

Source Publication Information

Journal: Experimental Technology and Management, Vol. 42, No. 9, September 2025, pp. 91–102. ISSN 1002-4956; CN 11-2034/T. DOI: 10.16791/j.cnki.sjg.2025.09.012.

Classification: Chinese Library Classification TP391.4; document code A; article number 1002-4956(2025)09-0091-12.

Manuscript dates: Received March 4, 2025; revised April 8, 2025.

Funding: China University of Mining and Technology, 2023 National Undergraduate Innovation Training Program (202310290032Z); China University of Mining and Technology Smart Course Development Project (2024KC19).

Author information: Xiaoping Yuan (born 1966), male, from Yizheng, Jiangsu; Ph.D., professor. His research focuses on signal processing, hardware acceleration of intelligent algorithms, and related areas. E-mail: xpyuankd@163.com.

Recommended citation: YUAN X P, TU Q Y. Image classification processing system based on FPGA and RK3588 [J]. Experimental Technology and Management, 2025, 42(9): 91–102. (in Chinese)

English Extended Abstract from the Source

Editorial note: The following English abstract is reproduced from the final page of the source, with typographical spacing normalized. Its numerical claims are retained as printed, not independently validated. Some conflict with the main text or figures; see the accompanying verification report.

Image classification processing system based on FPGA and RK3588

YUAN Xiaoping, TU Qiyang

(School of Information and Control Engineering, China University of Mining and Technology, Xuzhou 221116, China)

Abstract: [Objective] Vision transformer (ViT) models have achieved superior performance in image classification tasks owing to their global attention mechanisms. However, their high computational complexity and resource-intensiveness hinder their deployment on resource-constrained edge devices such as autonomous systems or robotics that demand low latency and high efficiency. Traditional CPU-based platforms pose issues such as prolonged processing delays, inefficient resource utilization, and limited parallel processing capabilities, struggling to meet the real-time requirements for ViT

inference. These limitations are exacerbated in scenarios requiring high frame rates (e.g., 60 Hz) and low power consumption. Therefore, a hybrid field-programmable gate array (FPGA) and neural processing unit (NPU) architecture was designed to accelerate ViT-based image classification while maintaining high accuracy and low latency. This system bridges the gap between the theoretical advantages of ViT and practical deployment constraints in edge computing environments.

[Methods] The system architecture integrated the FPGA and RK3588 NPU to optimize its computational efficiency via hardware codesign. The FPGA handled the image acquisition, preprocessing, and initial layers of the ViT model (e.g., patch embedding and positional encoding). Key optimizations included the following:

1. **Vectorized Computation:** The patch embedding layer was decomposed into vectorized operations, leveraging the parallel processing capabilities of the FPGA.
2. **Pipeline Design:** A pipelined architecture merged tensor operations for cumulative output, reducing intermediate data storage and latency.
3. **UDP-Based Data Transmission:** The FPGA-processed results were transmitted to the RK3588 NPU via the UDP protocol, ensuring real-time data flow and synchronizing tasks such as image display and classification.
4. **NPU Acceleration:** The RK3588 NPU employed multithreading and thread-pooling techniques for attention computation and deep-layer processing.

Second, the design workflow involved algorithm partitioning between the FPGA and RK3588 NPU, fixed-point quantization for FPGA compatibility, and rigorous simulation to validate time and resource constraints. For the FPGA implementation, model parameters were quantized to an eight-bit fixed-point format to reduce BRAM usage by 34% and maintain classification accuracy within 1.5% of the floating-point baseline. Hardware optimizations such as BRAM resource allocation and timing adjustments were implemented to eliminate setup violations and improve throughput. Postlayout simulations confirmed that the worst-case setup and hold times met the FPGA time margins (0.128 and 0.048 ns).

[Results] The experimental validation demonstrated considerable improvements across three key metrics:

1. **Latency Reduction:** The average per-frame processing time decreased from 0.6439 s (CPU baseline) to 0.0995 s, achieving real-time performance at 60 Hz, representing an 84.5% reduction in latency. This enabled seamless integration into time-sensitive applications such as autonomous navigation.

2. **Resource Efficiency:** FPGA BRAM utilization reached 56.42% with no timing violations after optimization. The design achieved a logic cell utilization of 72.3% and DSP slice utilization of 64.8%, demonstrating efficient resource allocation. The power consumption was measured at 3.2 W, considerably lower than that of GPU-based solutions (typically > 20 W).
3. **Classification Accuracy:** The system achieved average precision scores of 0.85 (human) and 0.905 (vehicle) on a custom dataset containing 10 000 annotated images. These results validated the effectiveness of hardware–software codesign, with a 1.2% accuracy degradation compared with the original floating-point model.

[Conclusions] By combining the parallel processing strengths of FPGA with high-throughput computing of NPU, this study successfully addressed the inefficiencies of ViT deployment on edge platforms. This architecture enhanced real time performance and maintained high accuracy, making it suitable for applications such as autonomous driving, industrial automation, and smart surveillance. The key innovations included the implementation of vectorized patch embedding, hybrid UDP–NPU task scheduling, and optimized fixed-point quantization. Future work will focus on expanding the model scalability to support large ViT variants (e.g., ViT-Large), exploring dynamic resource-sharing mechanisms between the FPGA and NPU, and integrating lightweight compression techniques to further reduce the memory footprint.

Key words: FPGA; RK3588; vectorized computation; superposition fusion; location identification message; multi-threaded design

Editor: Sun Hao.